

Network

네트워크

02 HTTP 프로토콜과 RESTful API 기본

목차 02 HTTP 프로토콜과 RESTful API 기본

1. 브라우저의 동작 원리
2. HTTP 프로토콜과 핵심 특징
3. HTTP 메시지와 HTTP 메서드
4. HTTP 요청 메시지/응답 메시지

목차 02 HTTP 프로토콜과 RESTful API 기본

5. HTTP 상태 코드

6. REST API 개요

7. REST API의 구성요소

1. 브라우저의 동작 원리



브라우저 주소창에 아래 주소를 입력하면 어떤 일이 일어날까?



<https://www.elice.io>



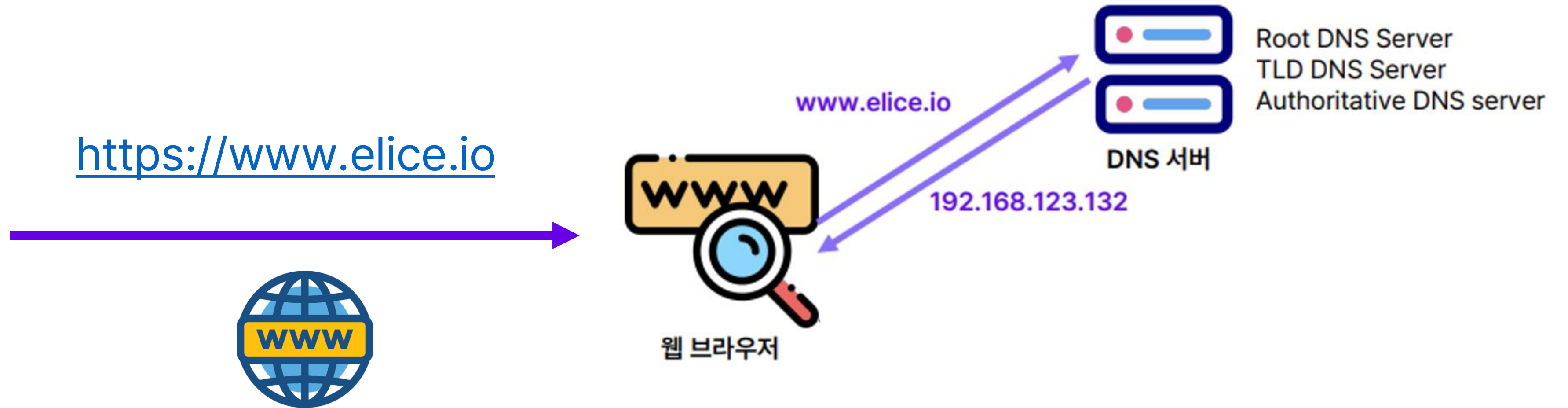
1. DNS 조회 -> IP 주소 반환

<https://www.elice.io>





1. DNS 조회 -> IP 주소 반환





2. 브라우저 HTTP 요청 메시지 생성



서버





2. 브라우저 HTTP 요청 메시지 생성



HTTP 요청 메시지 예시

GET / HTTP/1.1
Host: www.elice.io

--- 이하 생략 ---

서버





2. 브라우저 HTTP 요청 메시지 생성



	TCP/IP 모델
4계층	응용 계층
3계층	전송 계층
2계층	인터넷 계층
1계층	네트워크 액세스 계층

HTTP 요청 메시지 예시

GET / HTTP/1.1
Host: www.elice.io

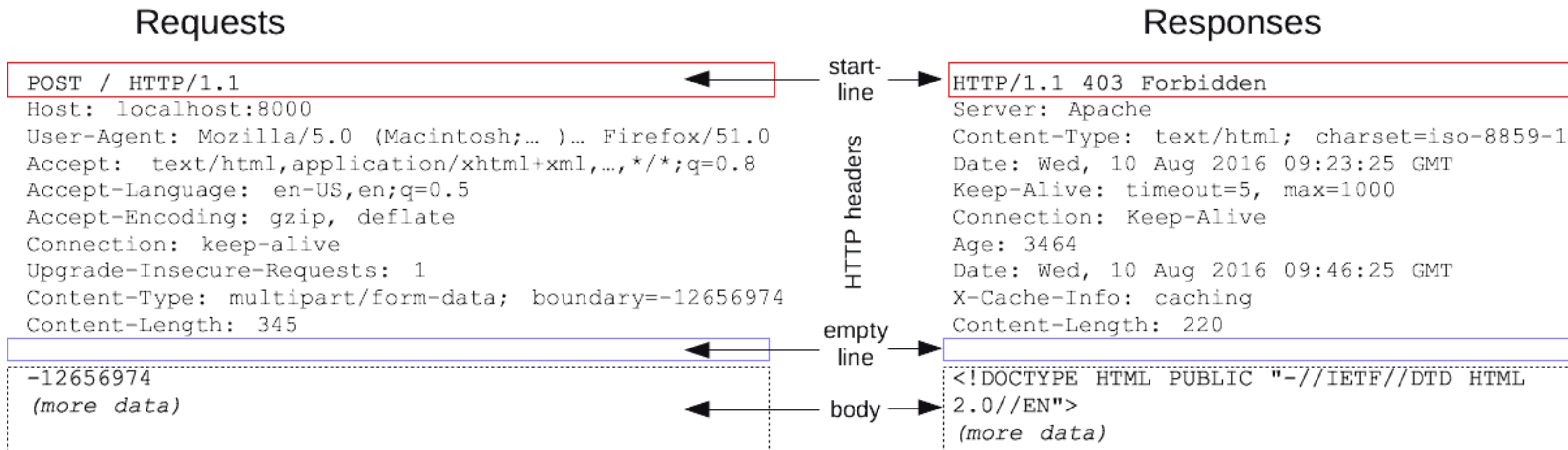
--- 이하 생략 ---

서버





2. 브라우저 HTTP 요청 메시지 생성



HTTP 메시지 기본 구조



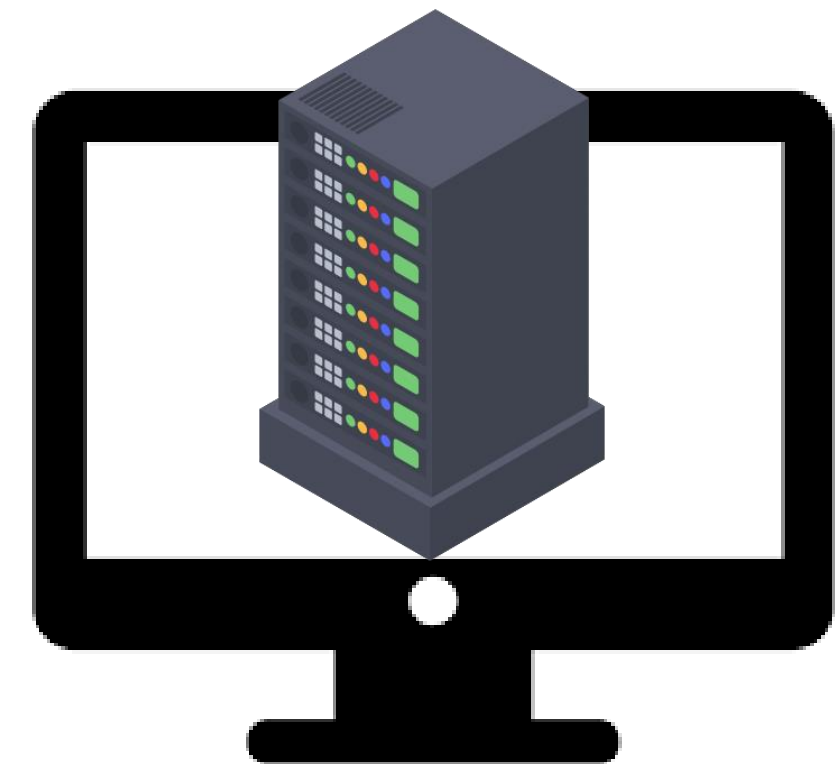
3. TCP/IP 패킷 생성



GET / HTTP/1.1
Host: www.elice.io

--- 이하 생략 ---

서버





3. TCP/IP 패킷 생성



IP 패킷 (출발지, 도착지 정보)

GET / HTTP/1.1
Host: www.elice.io

--- 이하 생략 ---

서버





3. TCP/IP 패킷 생성



	TCP/IP 모델
4계층	응용 계층
3계층	전송 계층
2계층	인터넷 계층
1계층	네트워크 액세스 계층

IP 패킷 (출발지, 도착지 정보)

GET / HTTP/1.1
Host: www.elice.io

--- 이하 생략 ---

서버





3. TCP/IP 패킷 생성

	TCP/IP 모델
4계층	응용 계층
3계층	전송 계층
2계층	인터넷 계층
1계층	네트워크 액세스 계층



HTTP 메시지

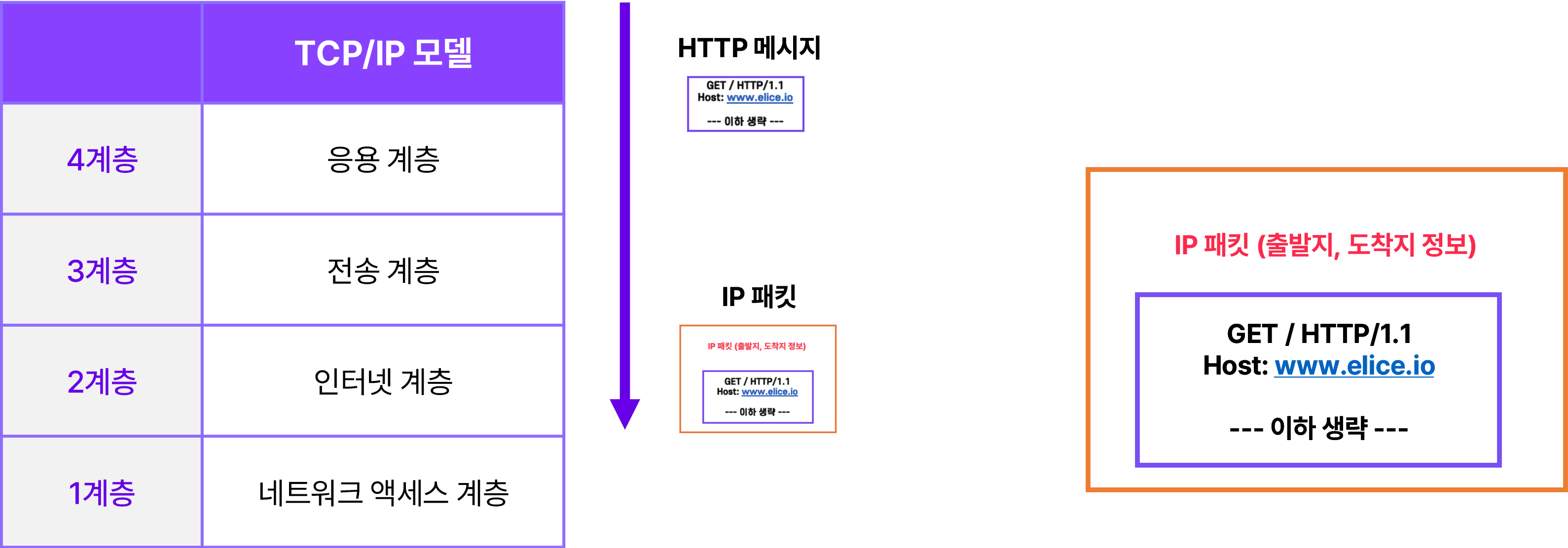
GET / HTTP/1.1
Host: www.elice.io
--- 이하 생략 ---

GET / HTTP/1.1
Host: www.elice.io

--- 이하 생략 ---

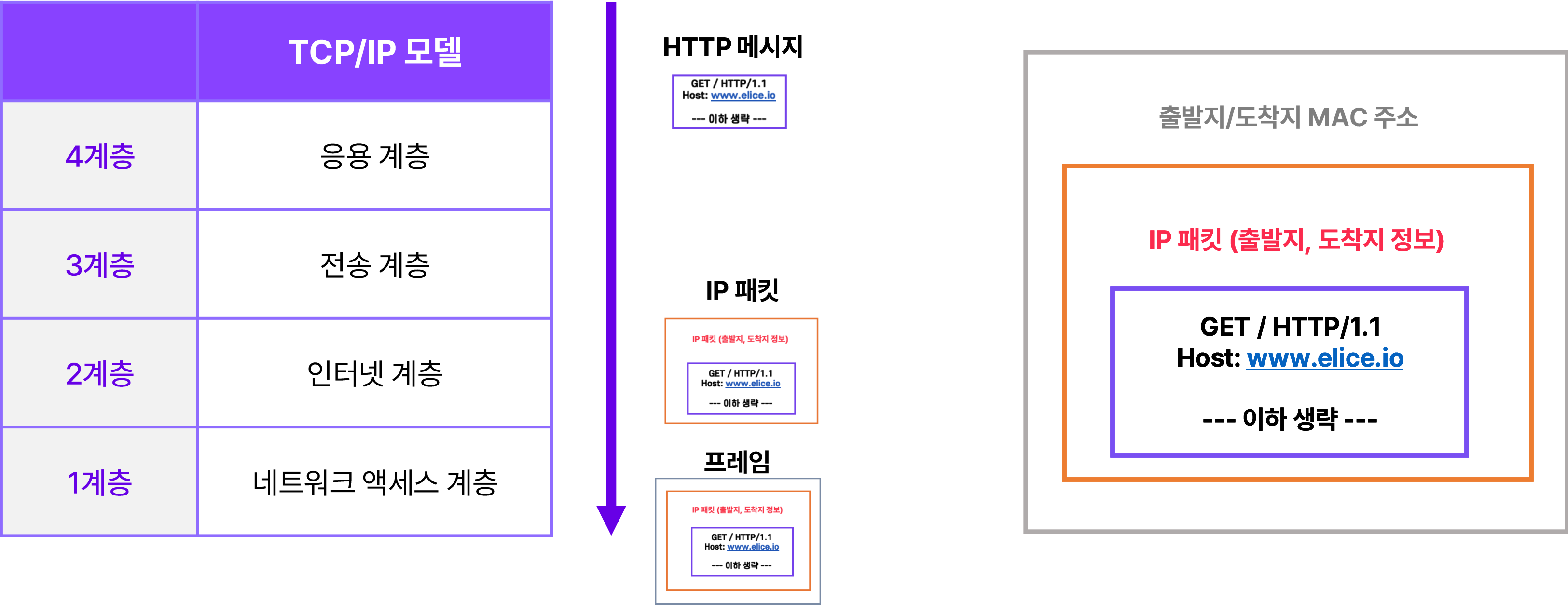


3. TCP/IP 패킷 생성





3. TCP/IP 패킷 생성





3. TCP/IP 패킷 생성

GET / HTTP/1.1
Host: www.elice.io
--- 이하 생략 ---



우편물 포장



3. TCP/IP 패킷 생성

GET / HTTP/1.1
Host: www.elice.io
--- 이하 생략 ---

IP 패킷 (출발지, 도착지 정보)

GET / HTTP/1.1
Host: www.elice.io
--- 이하 생략 ---



우편물 포장



3. TCP/IP 패킷 생성

GET / HTTP/1.1
Host: www.elice.io
--- 이하 생략 ---

IP 패킷 (출발지, 도착지 정보)

GET / HTTP/1.1
Host: www.elice.io
--- 이하 생략 ---

IP 패킷 (출발지, 도착지 정보)

GET / HTTP/1.1
Host: www.elice.io
--- 이하 생략 ---



우편물 포장





3. TCP/IP 패킷 생성

GET / HTTP/1.1
Host: www.elice.io
--- 이하 생략 ---

IP 패킷 (출발지, 도착지 정보)

GET / HTTP/1.1
Host: www.elice.io
--- 이하 생략 ---

IP 패킷 (출발지, 도착지 정보)

GET / HTTP/1.1
Host: www.elice.io
--- 이하 생략 ---

캡슐화
(Encapsulation)



우편물 포장



4. 요청 패킷 전달

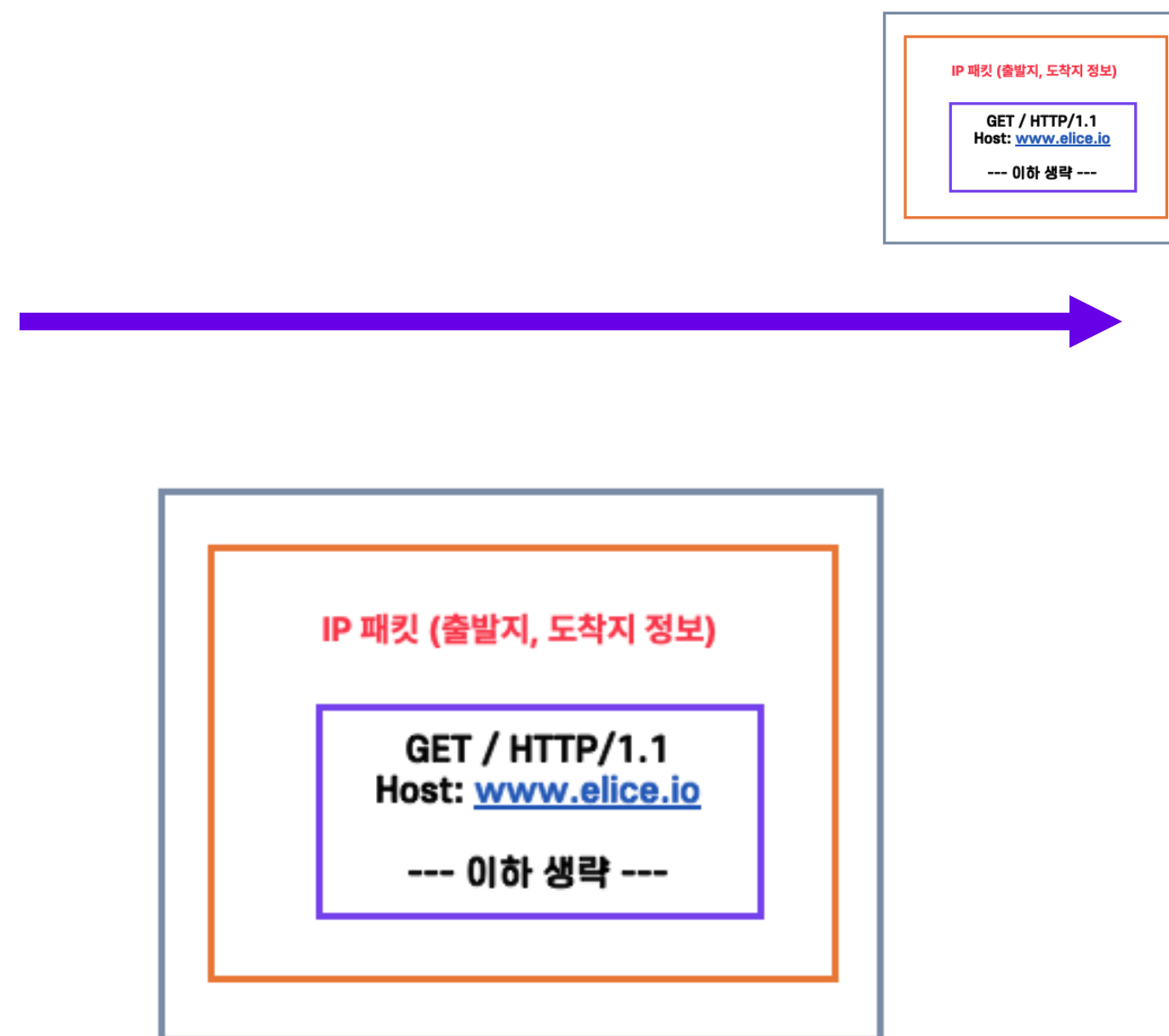


서버





4. 요청 패킷 전달

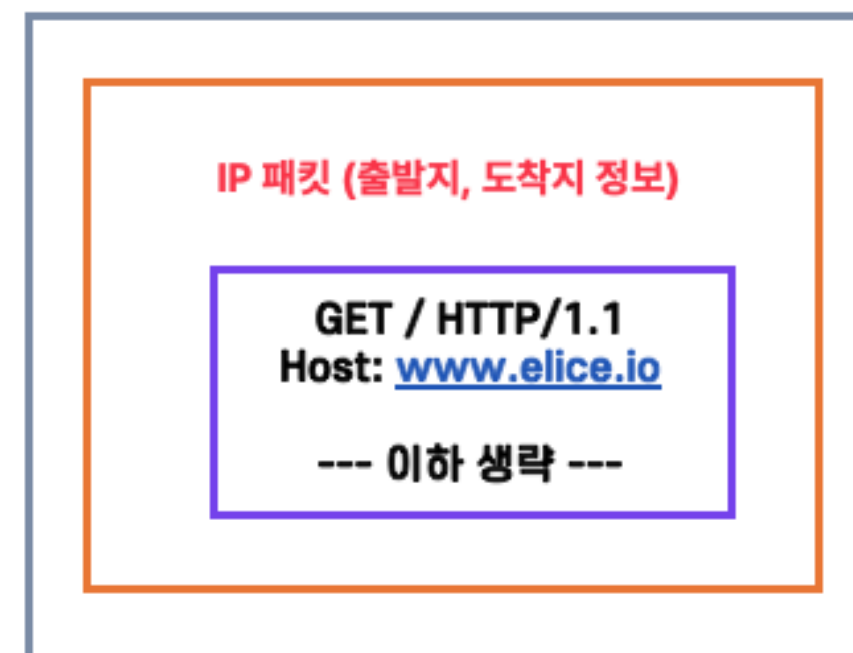


서버





5. TCP/IP 패킷 해체



우편 내용물 확인



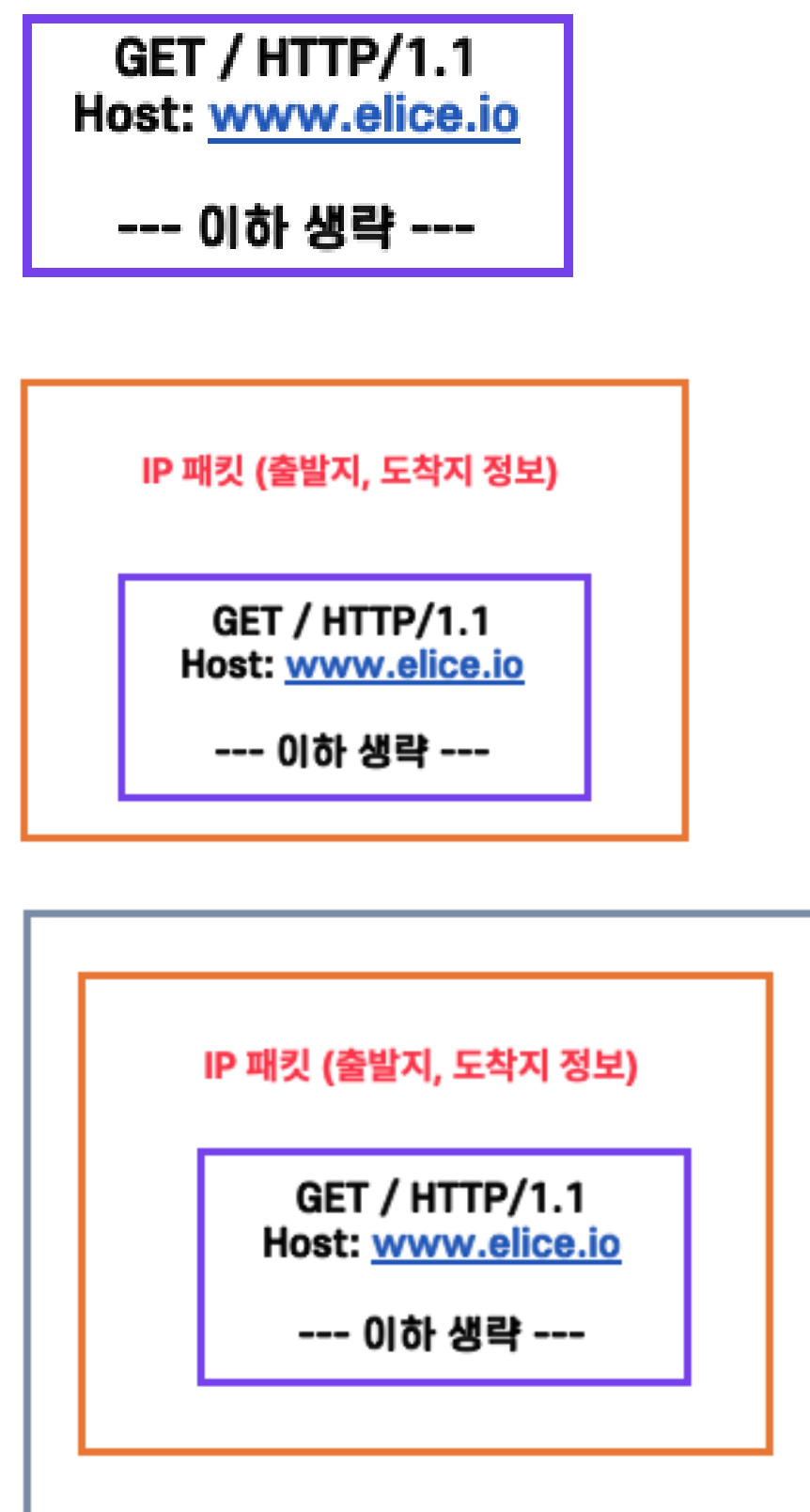
5. TCP/IP 패킷 해체



우편 내용물 확인



5. TCP/IP 패킷 해체



우편 내용물 확인



5. TCP/IP 패킷 해체

GET / HTTP/1.1
Host: www.elice.io
--- 이하 생략 ---

IP 패킷 (출발지, 도착지 정보)

GET / HTTP/1.1
Host: www.elice.io
--- 이하 생략 ---

IP 패킷 (출발지, 도착지 정보)

GET / HTTP/1.1
Host: www.elice.io
--- 이하 생략 ---

역캡슐화
(Decapsulation)



우편 내용물 확인





송신자 헬로빗



[송신 메시지]
커피 두 잔 주세요.

수신자 캐터필러



[수신 메시지]
커피 두 잔 주세요.

	TCP/IP 모델
4계층	응용 계층
3계층	전송 계층
2계층	인터넷 계층
1계층	네트워크 액세스 계층



	TCP/IP 모델
4계층	응용 계층
3계층	전송 계층
2계층	인터넷 계층
1계층	네트워크 액세스 계층



송신자 헬로빗



[송신 메시지]
커피 두 잔 주세요.

수신자 캐터필러



[수신 메시지]
커피 두 잔 주세요.

	TCP/IP 모델
4계층	응용 계층
3계층	전송 계층
2계층	인터넷 계층
1계층	네트워크 액세스 계층



GET / HTTP/1.1
Host: www.elfice.io
--- 이하 생략 ---

IP 패킷 (출발지, 도착지 정보)
GET / HTTP/1.1
Host: www.elfice.io
--- 이하 생략 ---

IP 패킷 (출발지, 도착지 정보)
GET / HTTP/1.1
Host: www.elfice.io
--- 이하 생략 ---

	TCP/IP 모델
4계층	응용 계층
3계층	전송 계층
2계층	인터넷 계층
1계층	네트워크 액세스 계층



송신자 헬로빗



[송신 메시지]
커피 두 잔 주세요.

수신자 캐터필러



[수신 메시지]
커피 두 잔 주세요.

	TCP/IP 모델
4계층	응용 계층
3계층	전송 계층
2계층	인터넷 계층
1계층	네트워크 액세스 계층



	TCP/IP 모델
4계층	응용 계층
3계층	전송 계층
2계층	인터넷 계층
1계층	네트워크 액세스 계층



송신자 헬로빗



[송신 메시지]
커피 두 잔 주세요.

수신자 캐터필러



[수신 메시지]
커피 두 잔 주세요.

	TCP/IP 모델
4계층	응용 계층
3계층	전송 계층
2계층	인터넷 계층
1계층	네트워크 액세스 계층



	TCP/IP 모델
4계층	응용 계층
3계층	전송 계층
2계층	인터넷 계층
1계층	네트워크 액세스 계층



송신자 헬로빗



[송신 메시지]
커피 두 잔 주세요.

수신자 캐터필러



[수신 메시지]
커피 두 잔 주세요.

	TCP/IP 모델
4계층	응용 계층
3계층	전송 계층
2계층	인터넷 계층
1계층	네트워크 액세스 계층



	TCP/IP 모델
4계층	응용 계층
3계층	전송 계층
2계층	인터넷 계층
1계층	네트워크 액세스 계층

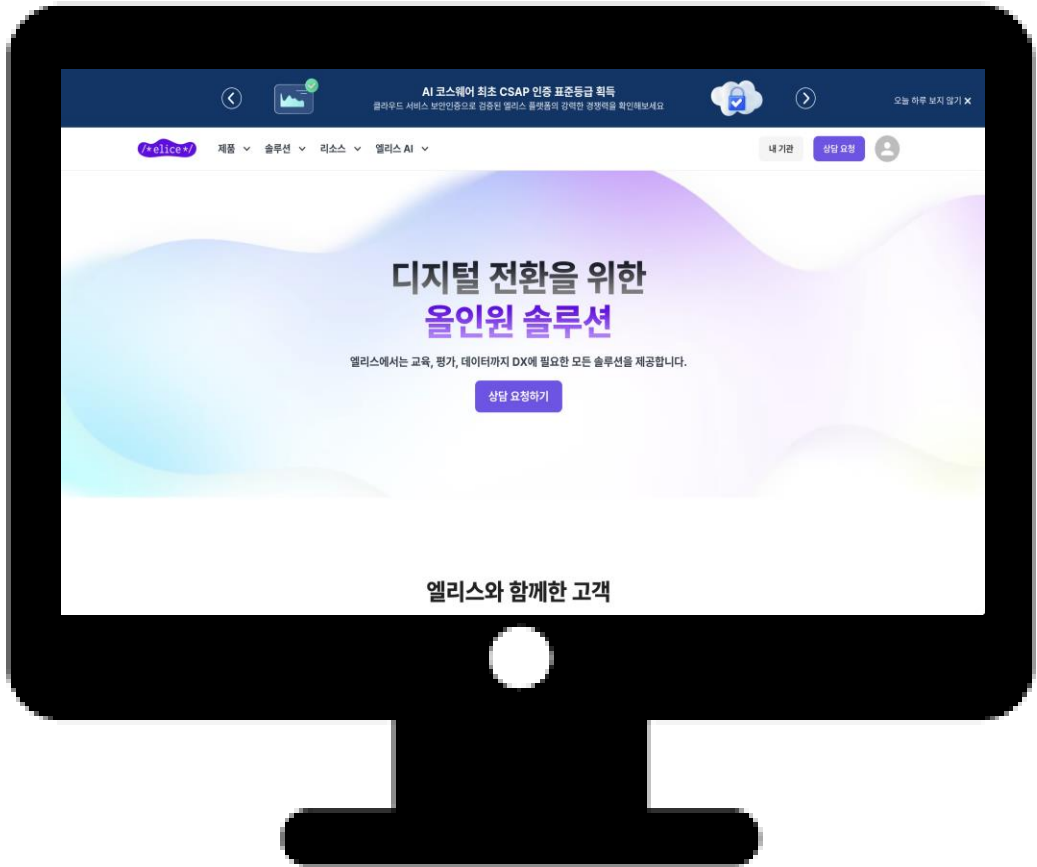


6. 서버의 응답 패킷 전달





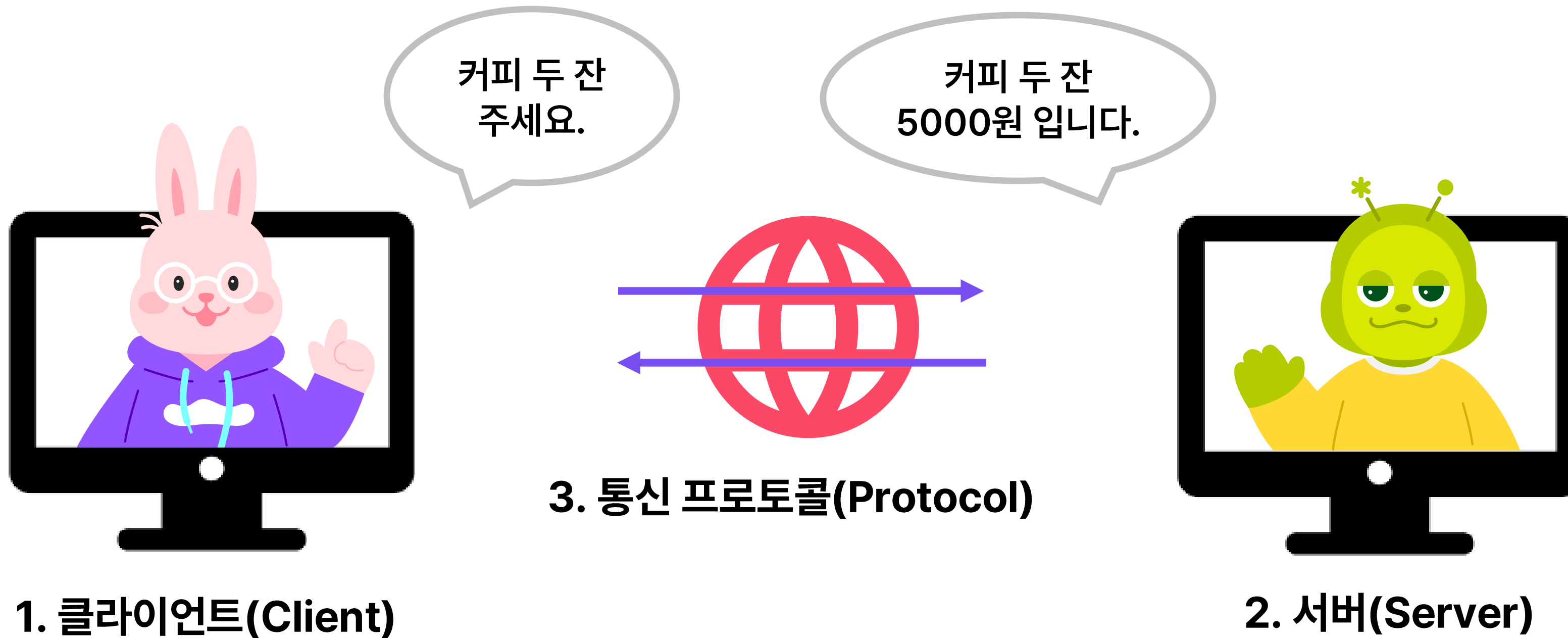
6. 서버의 응답 패킷 전달



서버



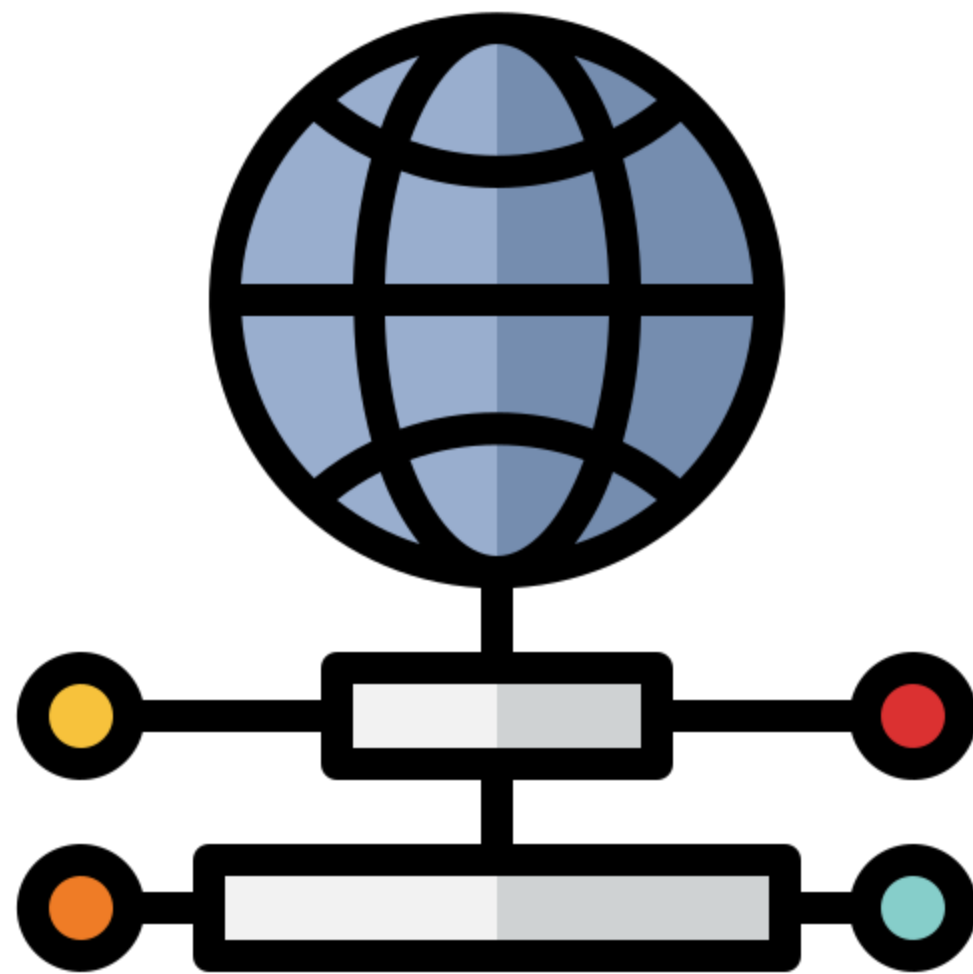
2. HTTP 프로토콜과 핵심 특징





통신의 과정에 필요한 형식, 약속, 규칙 등을 정의 한 것

“프로토콜(Protocol)”



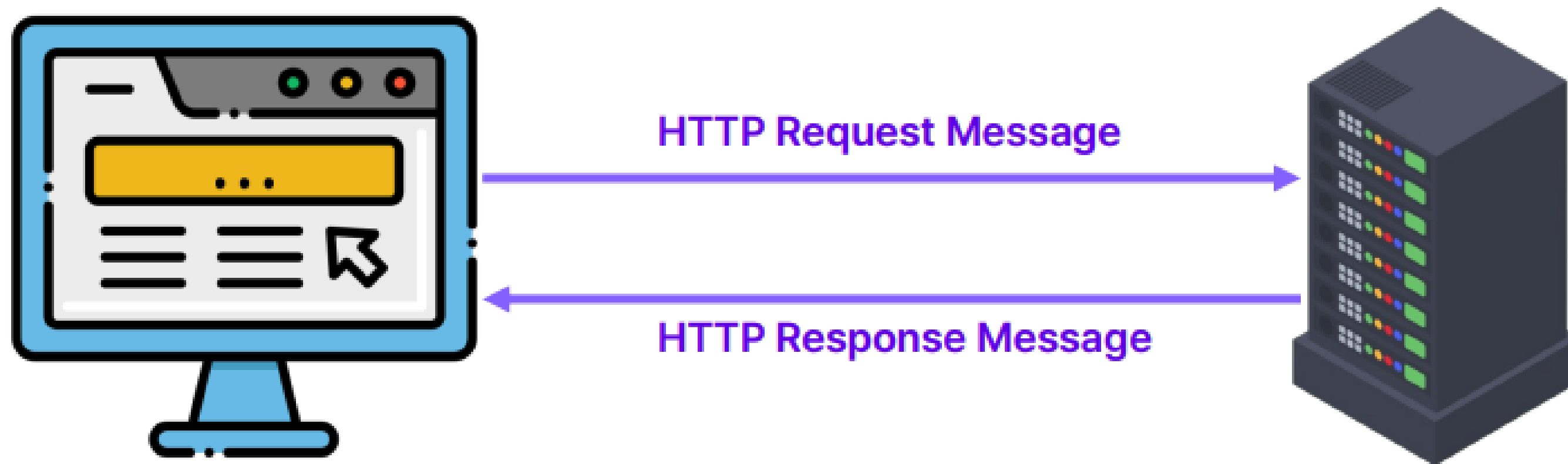
- 네트워크 통신에서 **데이터를 교환하기 위한 규칙과 표준** 제공
- 데이터 전송의 일관성 보장
- 다양한 기기와 시스템 간의 상호운용성 제공
- 오류 검출 및 수정 기능 제공



기능	역할
단편화/재합성	데이터 블록을 나누고 합쳐서 전송
캡슐화	프로토콜에 필요한 데이터 정보를 부착
연결/흐름/오류 제어	데이터 전송에 필요한 연결/흐름/오류를 개설·조정·종결
순서 결정	데이터 단위 순서대로 수신 측에 전달
주소 설정	발생지, 목적지 등의 주소를 명기하여 데이터를 전달
동기화/다중화	상태를 일치시키는 기능/통신로를 나누어 동시에 사용할 수 있도록 하는 가능



계층	프로토콜
응용 (Application)	HTTP, SMTP, FTP, Telnet
표현 (Presentation)	ASCII, MPEG, JPEG, MIDI
세션 (Session)	NetBIOS, SAP, SDP, NWLink
전송 (Transport)	TCP, UDP, SPX
네트워크 (Network)	IP, IPX
데이터 링크 (Data Link)	Ethernet, Token Ring, FDDI, AppleTalk
물리 (Physical)	없음



- **H**yper**T**ext **T**ransfer **P**rotocol
- 클라이언트-서버 간 데이터가 교환되는 방식에 대한 규약
- 거의 모든 형태의 데이터 전송 가능
- 무상태성과 비연결성

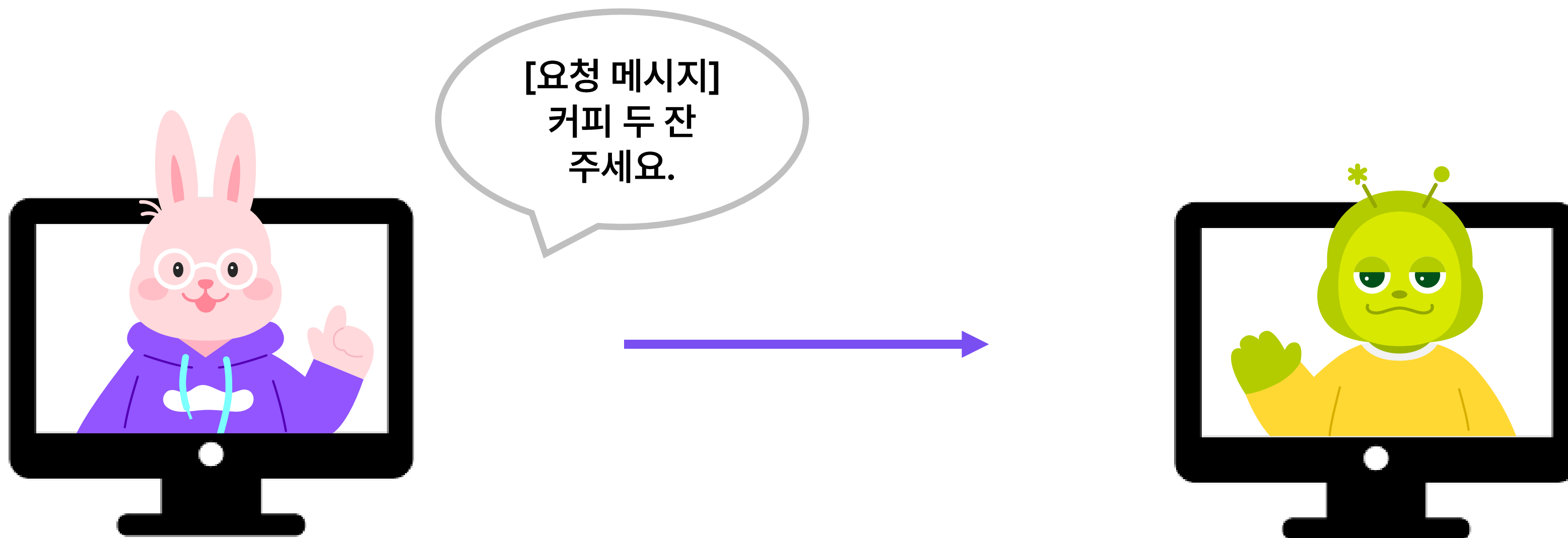


무상태성 (Stateless)

- 서버가 클라이언트의 **상태를 보존하지 않음**
- 각 요청은 독립적이며, 이전 요청과의 연결성이 존재하지 않음
- 서버의 확장성에 용이

비연결성(Connectionless)

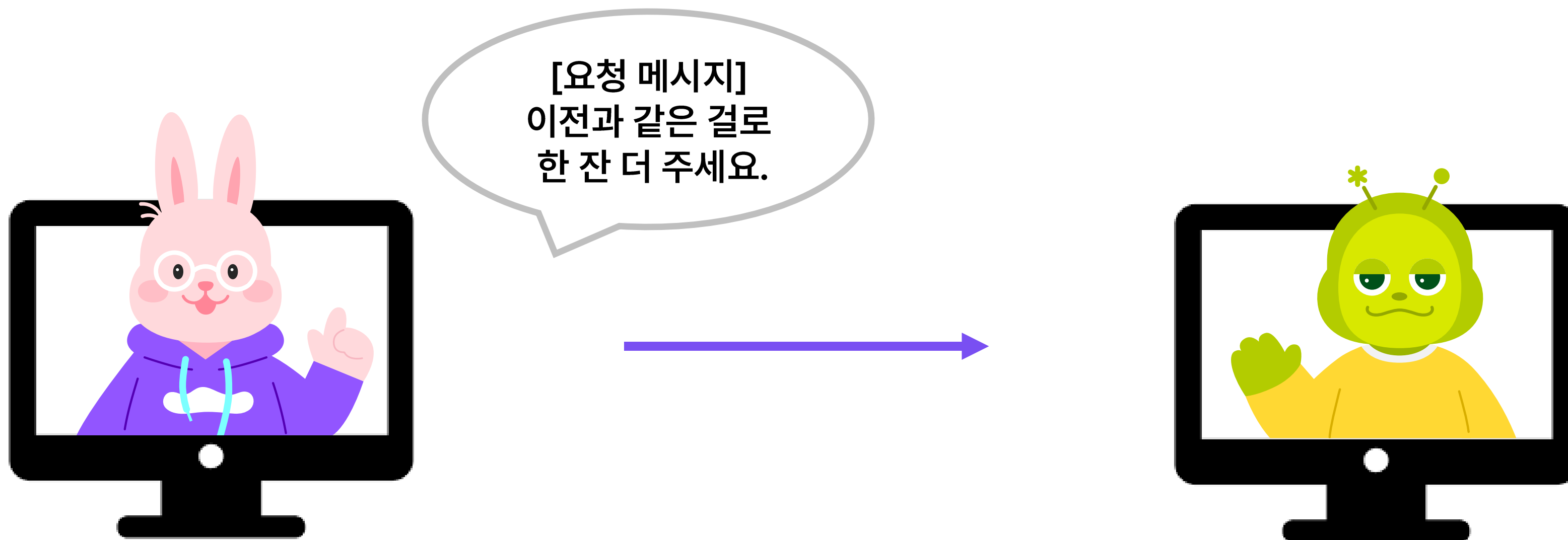
- 클라이언트-서버 간 **연결을 유지하지 않음**
- 서버의 불필요한 연결을 줄이고 많은 클라이언트의 요청을 동시 처리 가능





[응답 메시지]
커피 두 잔
나왔습니다.

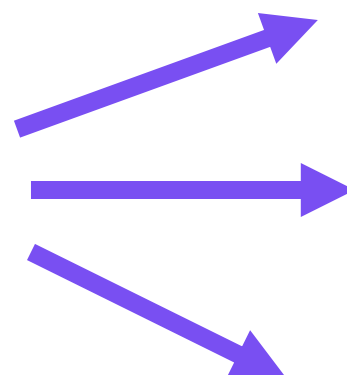




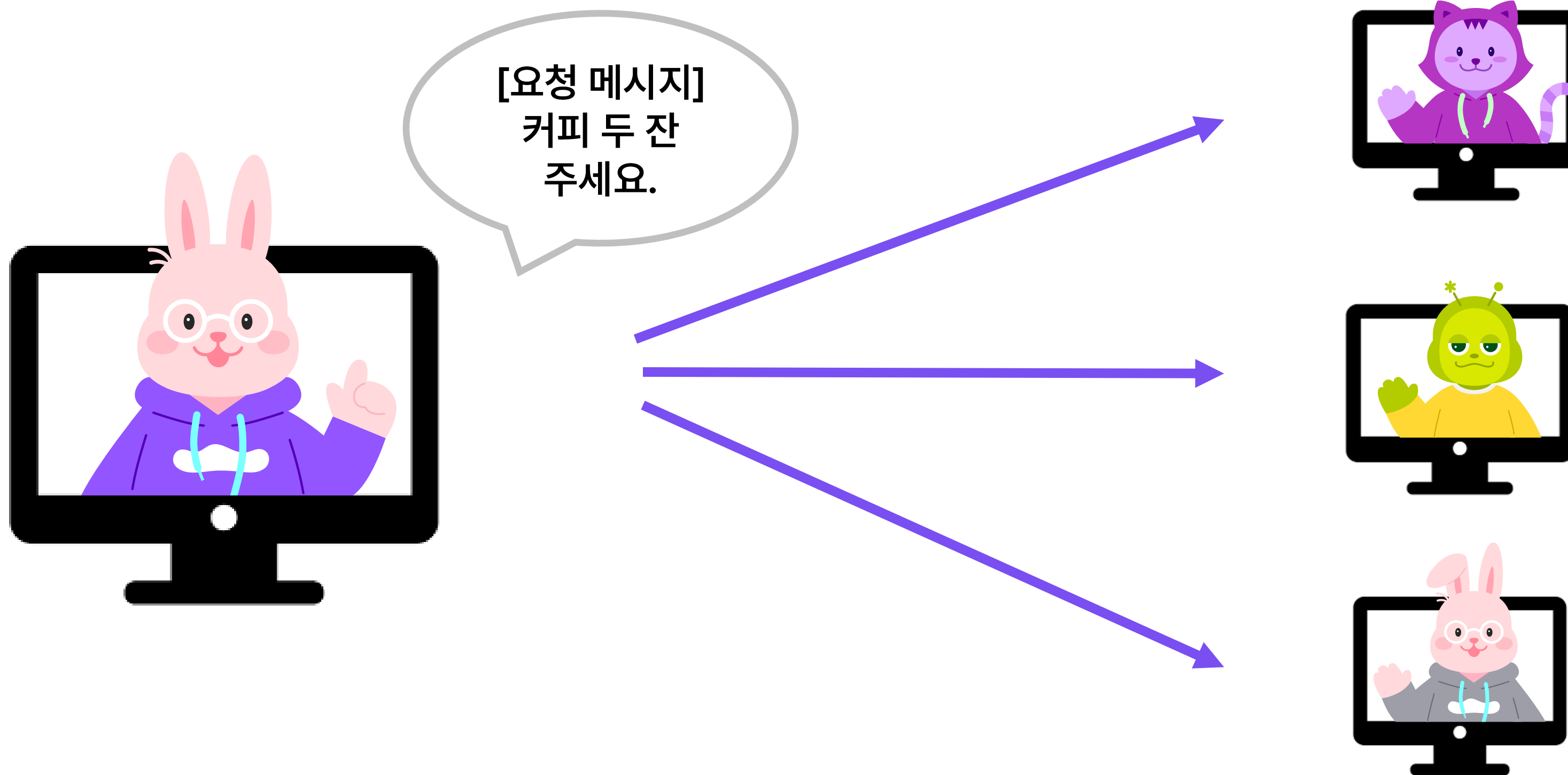


[응답 메시지]
이전 거요??

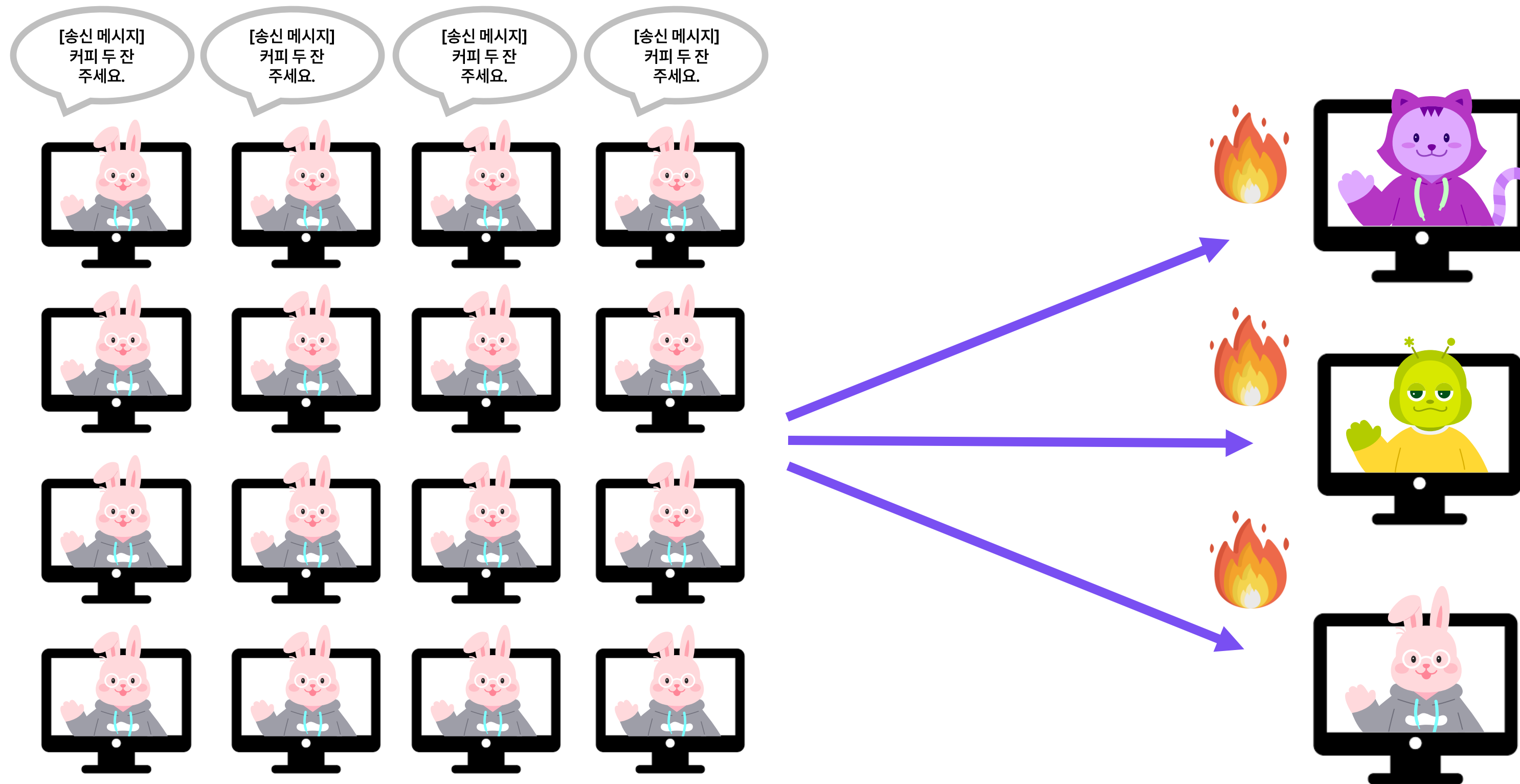




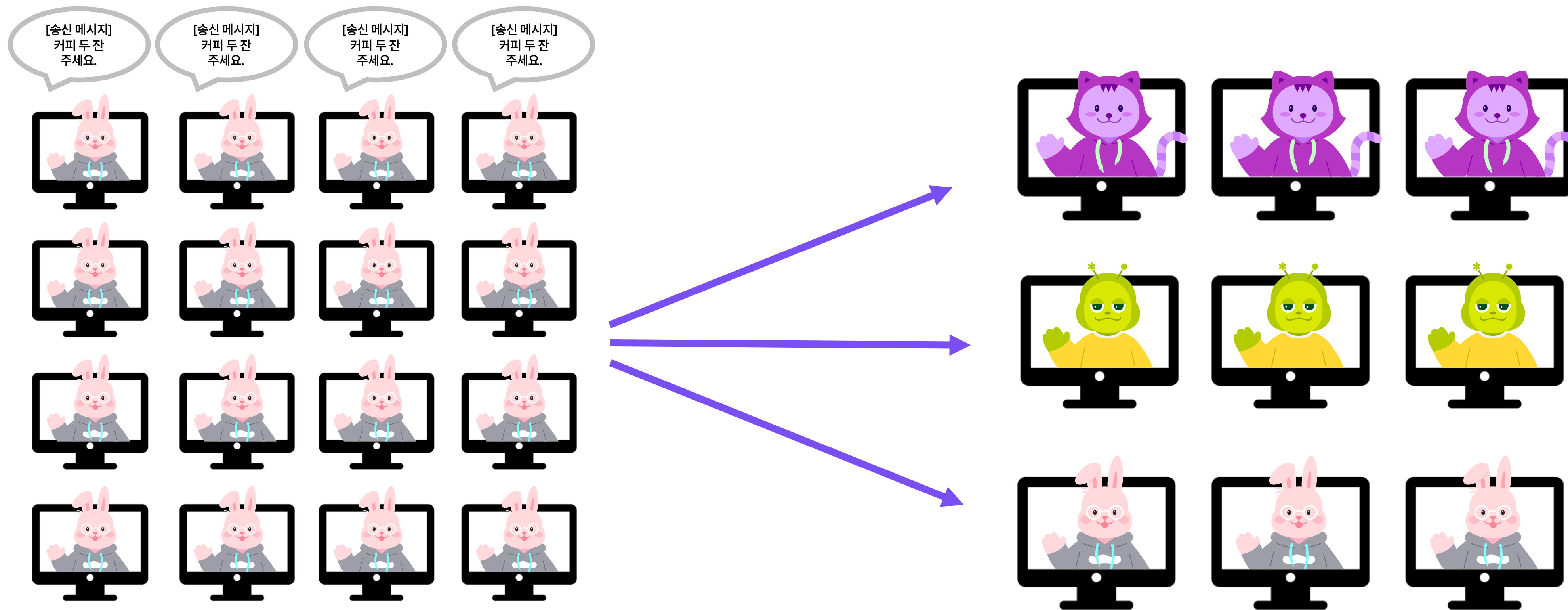
아무에게나 편하게 요청 가능



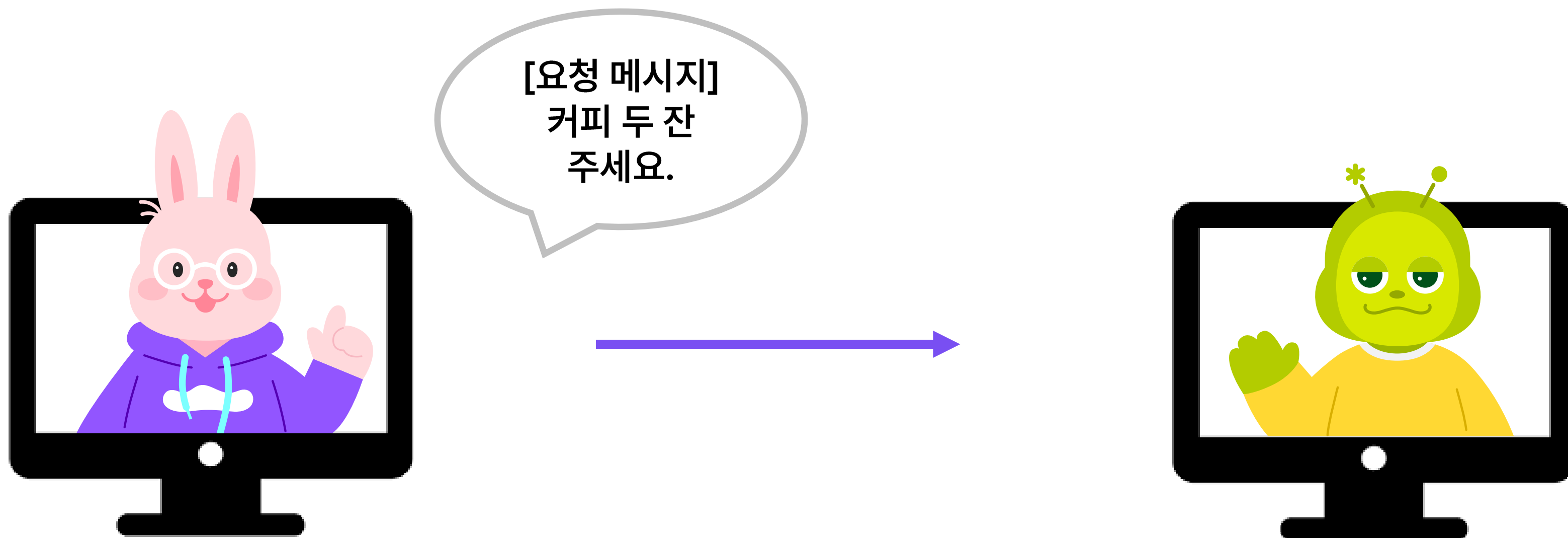
아무에게나 편하게 요청 가능

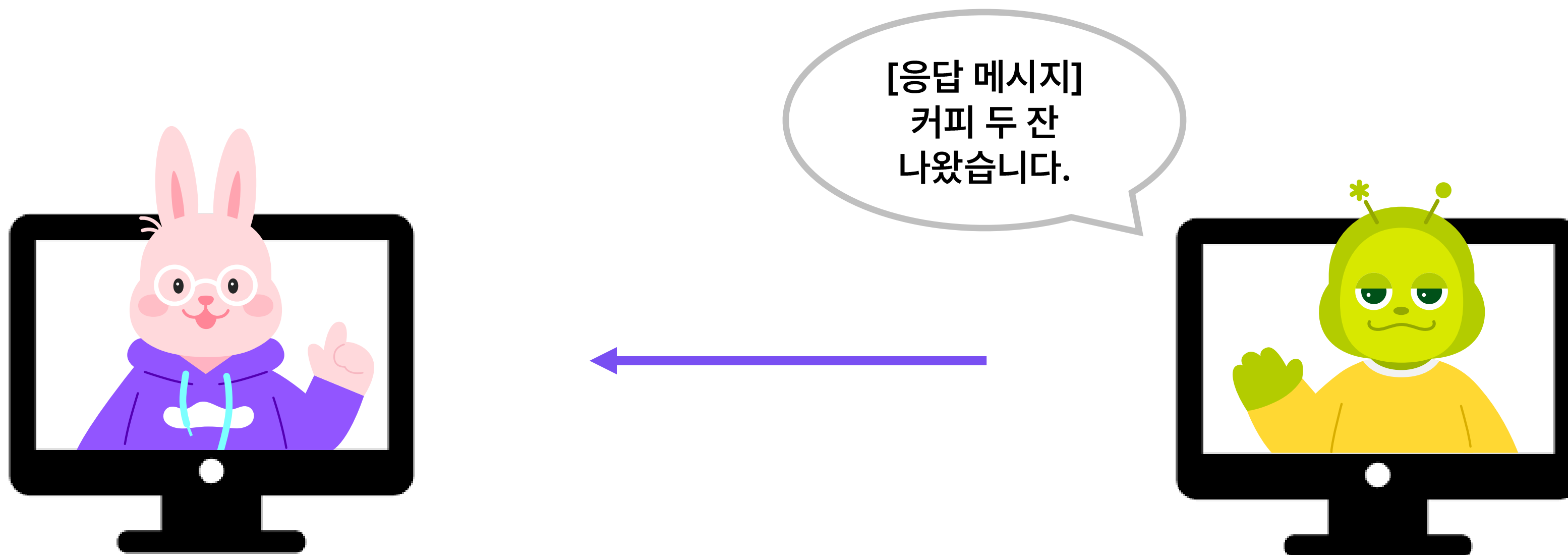


요청이 많아지면 단순히 점원의 수를 늘려서 문제 해결 가능



요청이 많아지면 단순히 점원의 수를 늘려서 문제 해결 가능







요청 - 응답 후 연결 종료 (연결 지속 X)



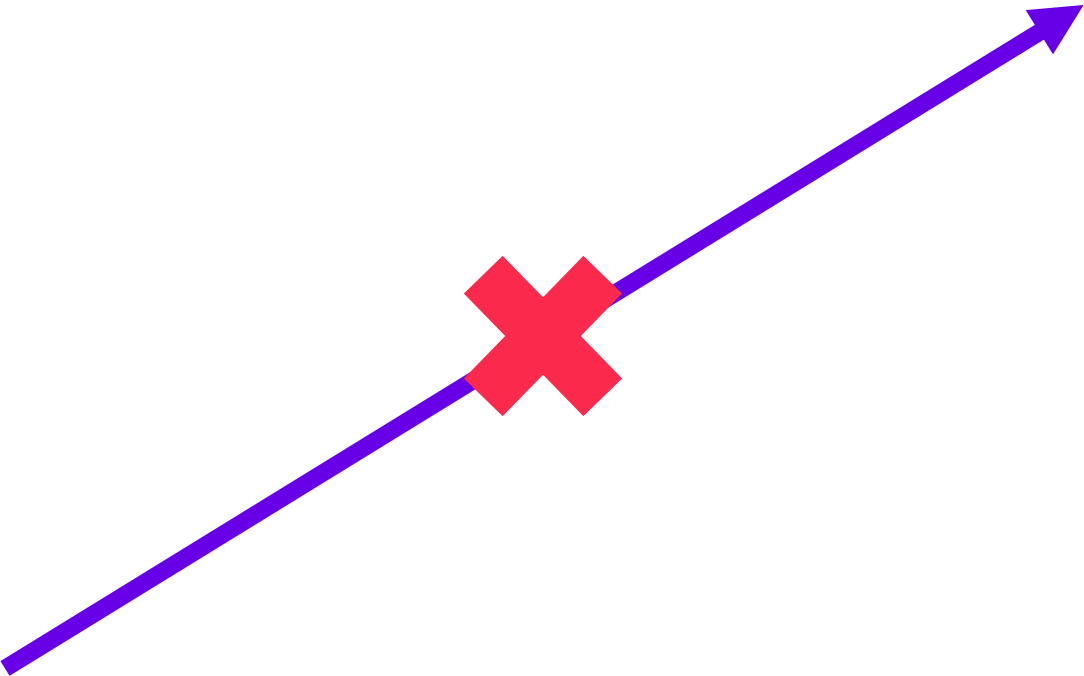
연결 지속



연결이 지속되는 경우, 다른 요청 응답 어려움



연결 지속



연결이 지속되는 경우, 다른 요청 응답 어려움

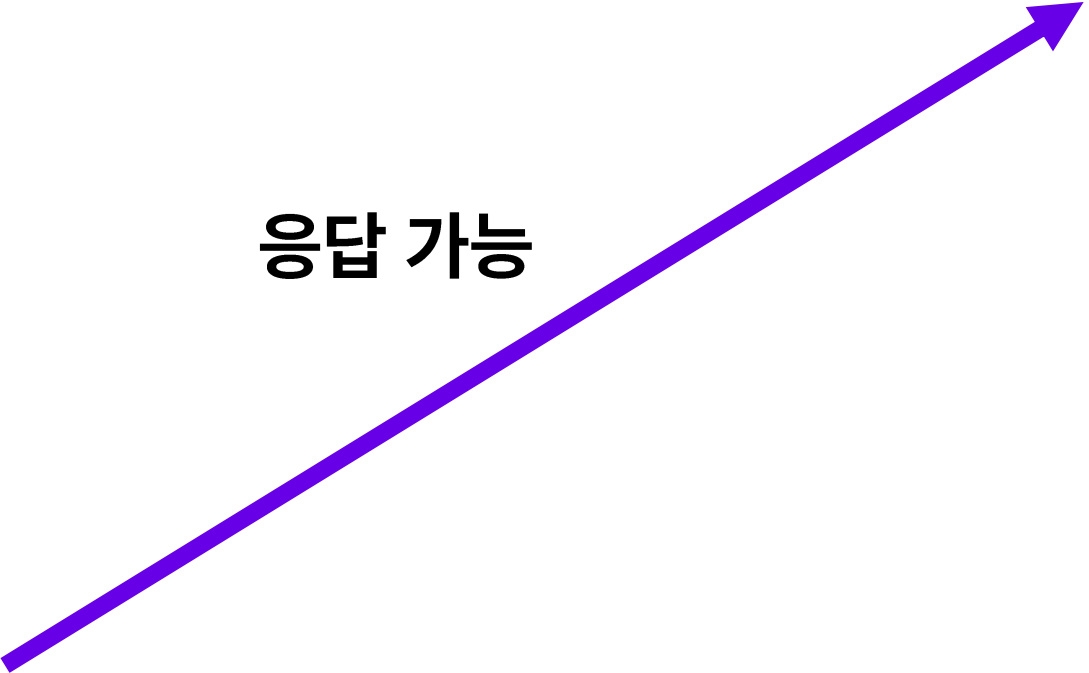


요청-응답 후 연결 해제



서버 자원의 효율화

응답 가능





무상태성 (Stateless)

- 서버가 클라이언트의 **상태를 보존하지 않음**
- 각 요청은 독립적이며, 이전 요청과의 연결성이 존재하지 않음

비연결성(Connectionless)

- 클라이언트-서버 간 **연결을 유지하지 않음**
- 서버의 불필요한 연결을 줄이고 많은 클라이언트의 요청을 동시 처리 가능



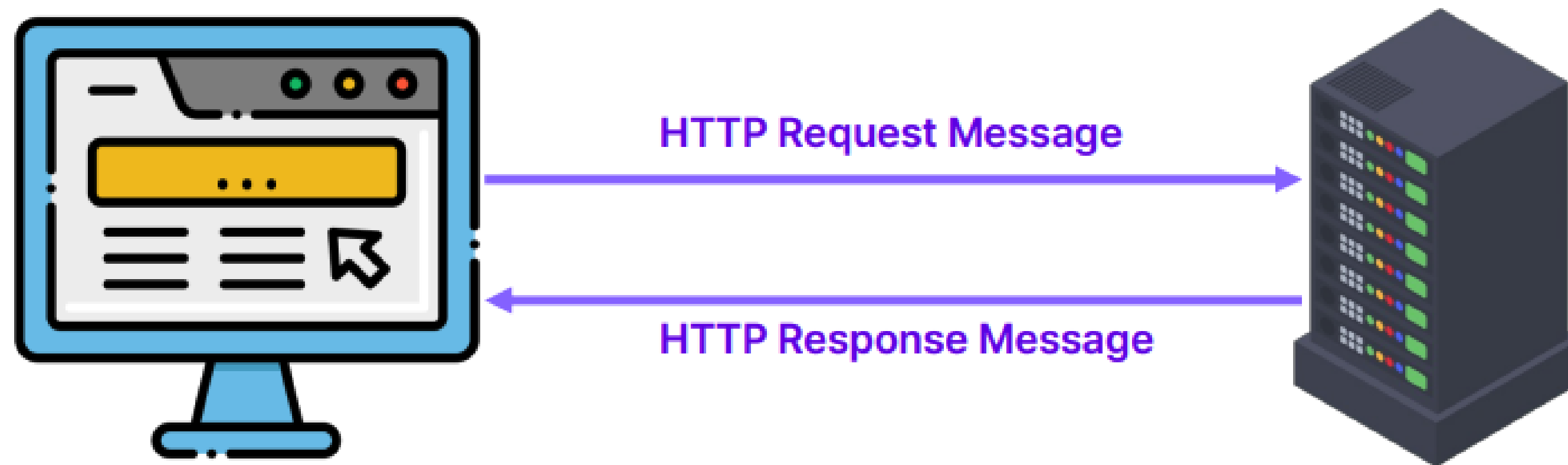
무상태성 (Stateless)

- 서버가 클라이언트의 **상태를 보존하지 않음**
- 각 요청은 독립적이며, 이전 요청과의 연결성이 존재하지 않음

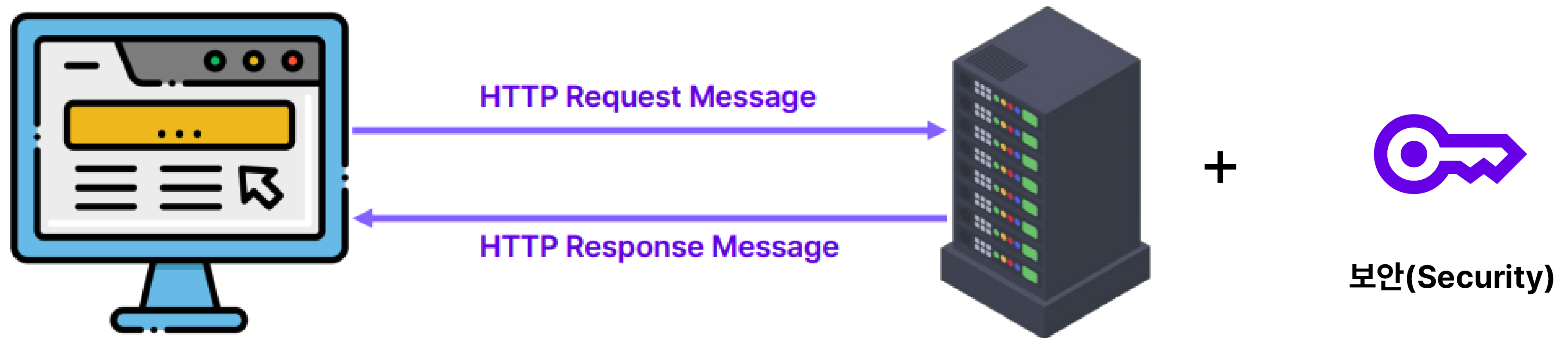
비연결성(Connectionless)

- 클라이언트-서버 간 **연결을 유지하지 않음**
- 서버의 불필요한 연결을 줄이고 많은 클라이언트의 요청을 동시 처리 가능

- 
- 서버의 확장성에 유리
 - 단순한 구현
 - 서버 자원의 효율화

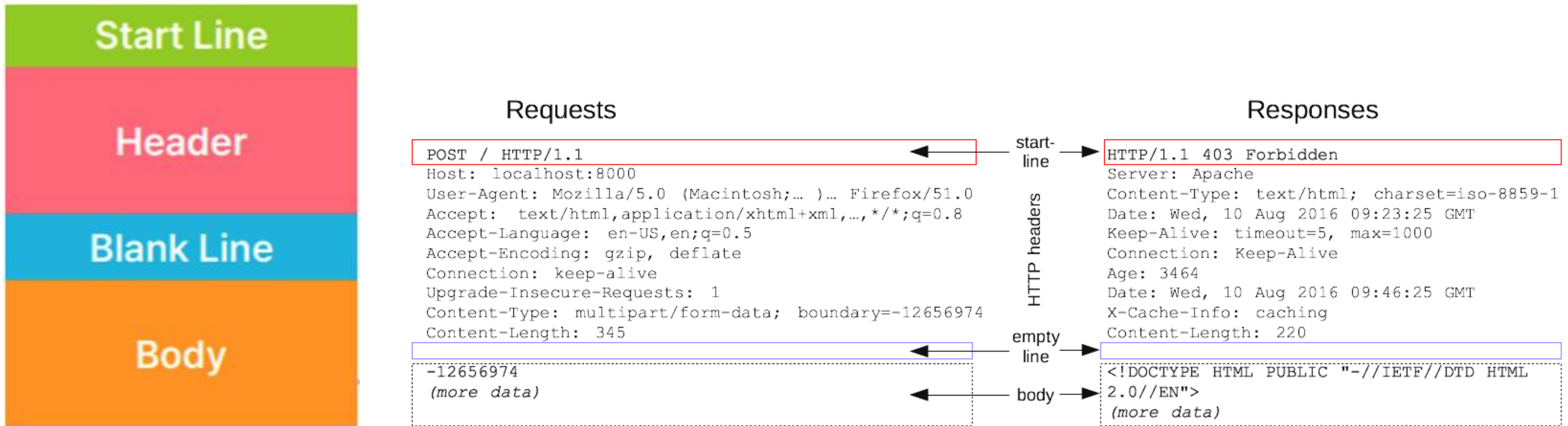


- **HTTP + S(Secure)** => HTTP의 보안 버전
- 데이터 암호화 -> 데이터 보호 및 신뢰성 향상
- TLS/SSL 프로토콜 사용

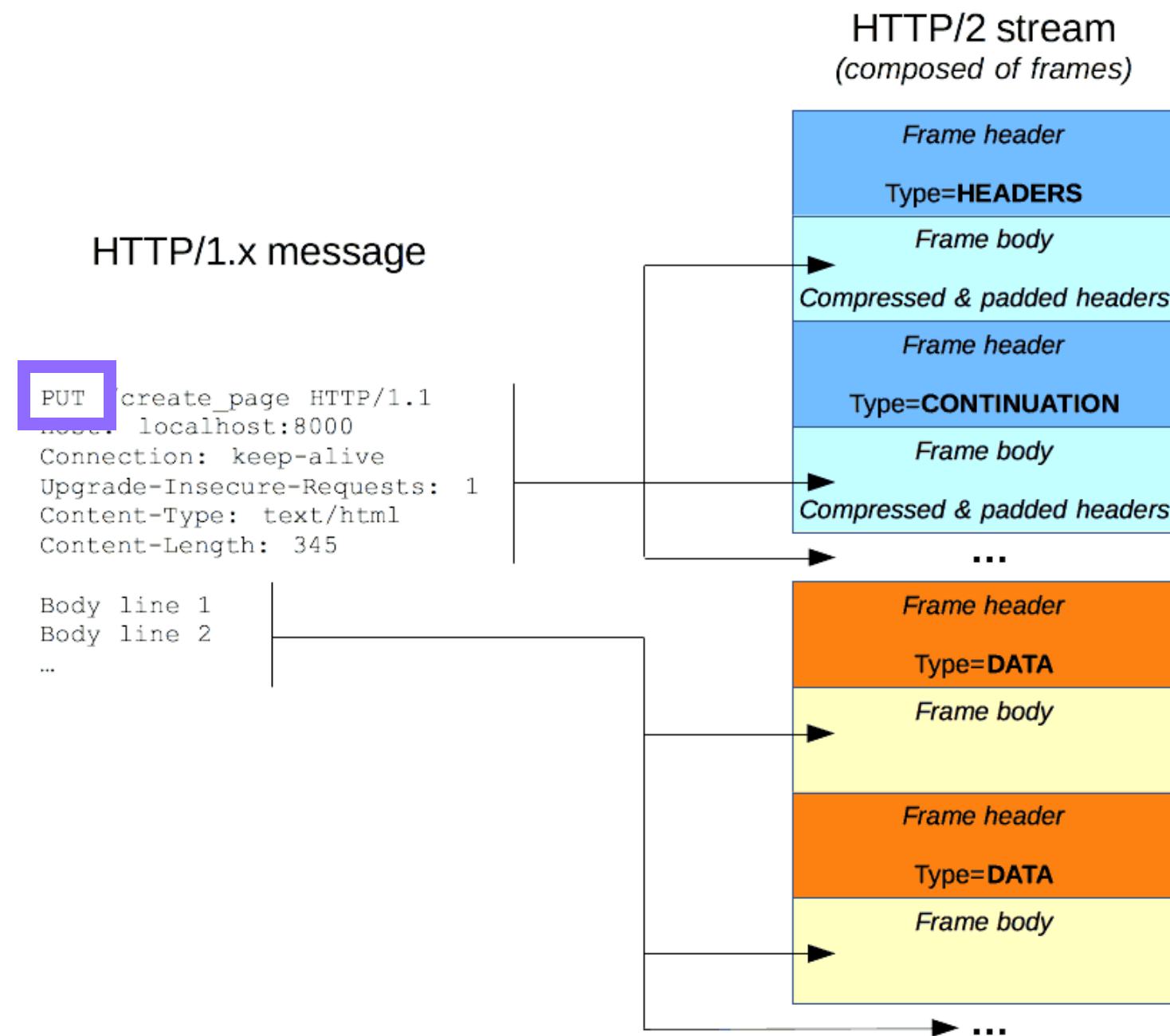


- **HTTP + S(Secure)** => HTTP의 보안 버전
- 데이터 암호화 -> 데이터 보호 및 신뢰성 향상
- TLS/SSL 프로토콜 사용

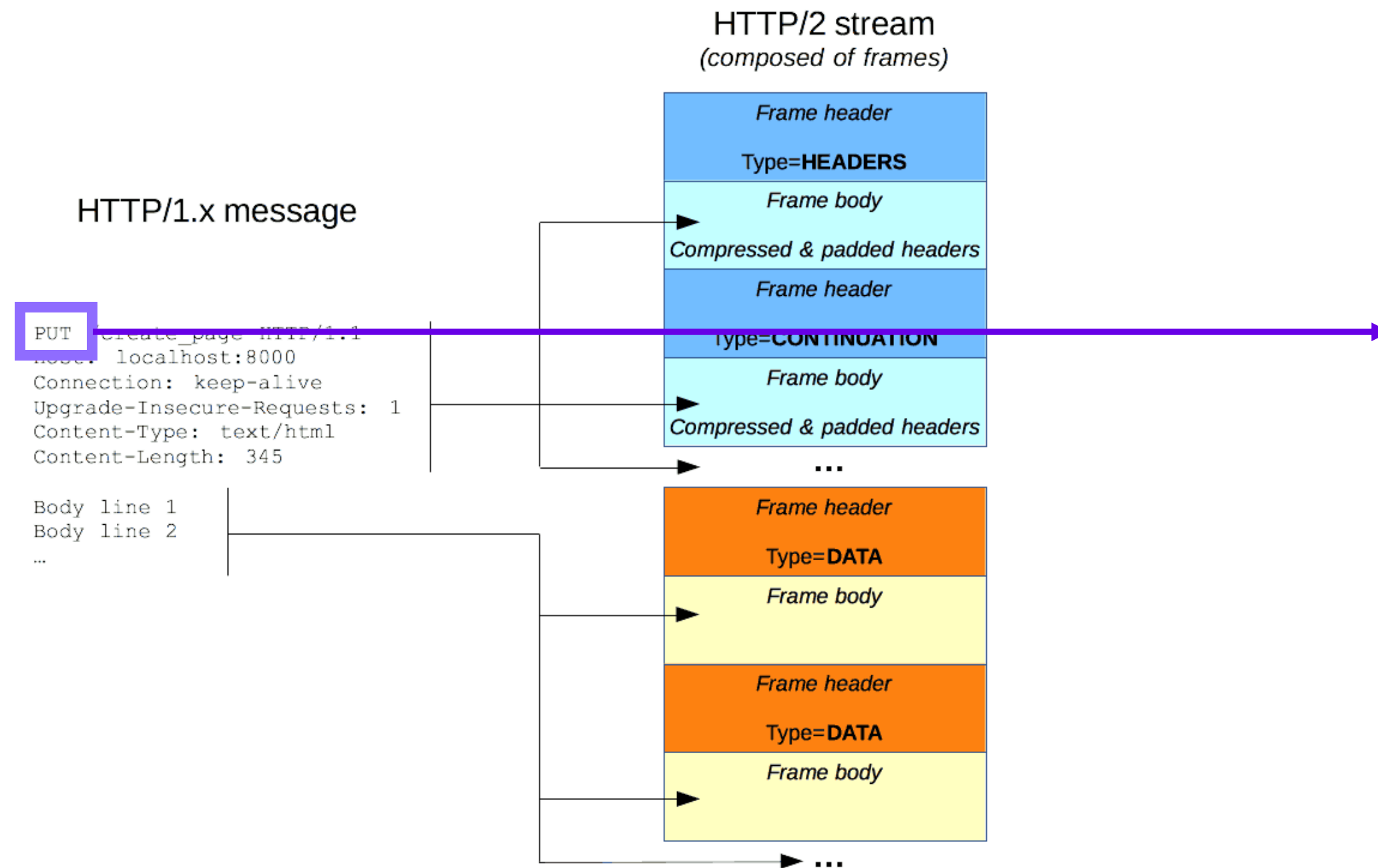
3. HTTP 메시지와 HTTP 메서드



- 웹 상에서 클라이언트-서버 간 데이터 전송 형식을 규정
- 시작 라인(Start Line), 헤더(Header), 공백(Blank Line), 본문(Body)으로 구성
- 요청 메시지/응답 메시지에 따라 담기는 내용의 차이 존재



- 클라이언트가 서버에게 수행할 **작업의 종류**를 지시하는 **명령어**
- RESTful API 설계에서 중요한 역할

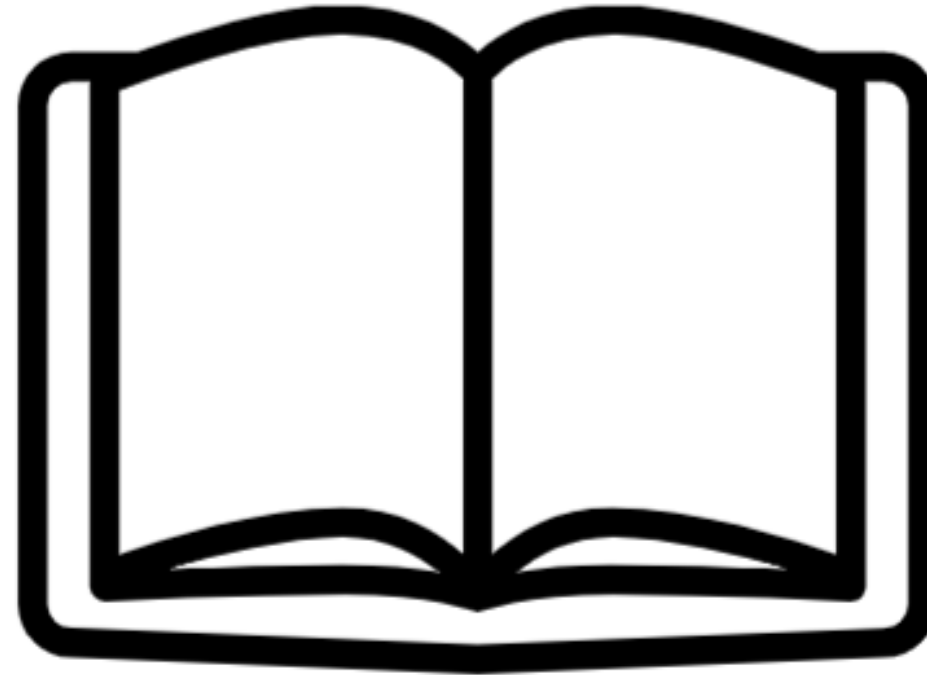


메서드	역할
GET	자료를 조회할 때 사용
POST	자료를 등록할 때 사용
PUT	자료의 수정을 요청할 때 사용
Delete	자료의 삭제를 요청할 때 사용

- 클라이언트가 서버에게 수행할 작업의 종류를 지시하는 명령어
- RESTful API 설계에서 중요한 역할



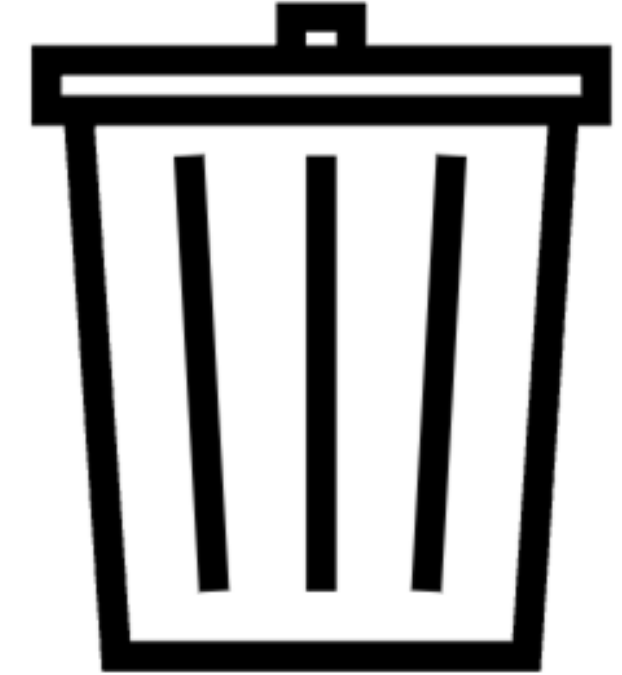
C
Create



R
Read

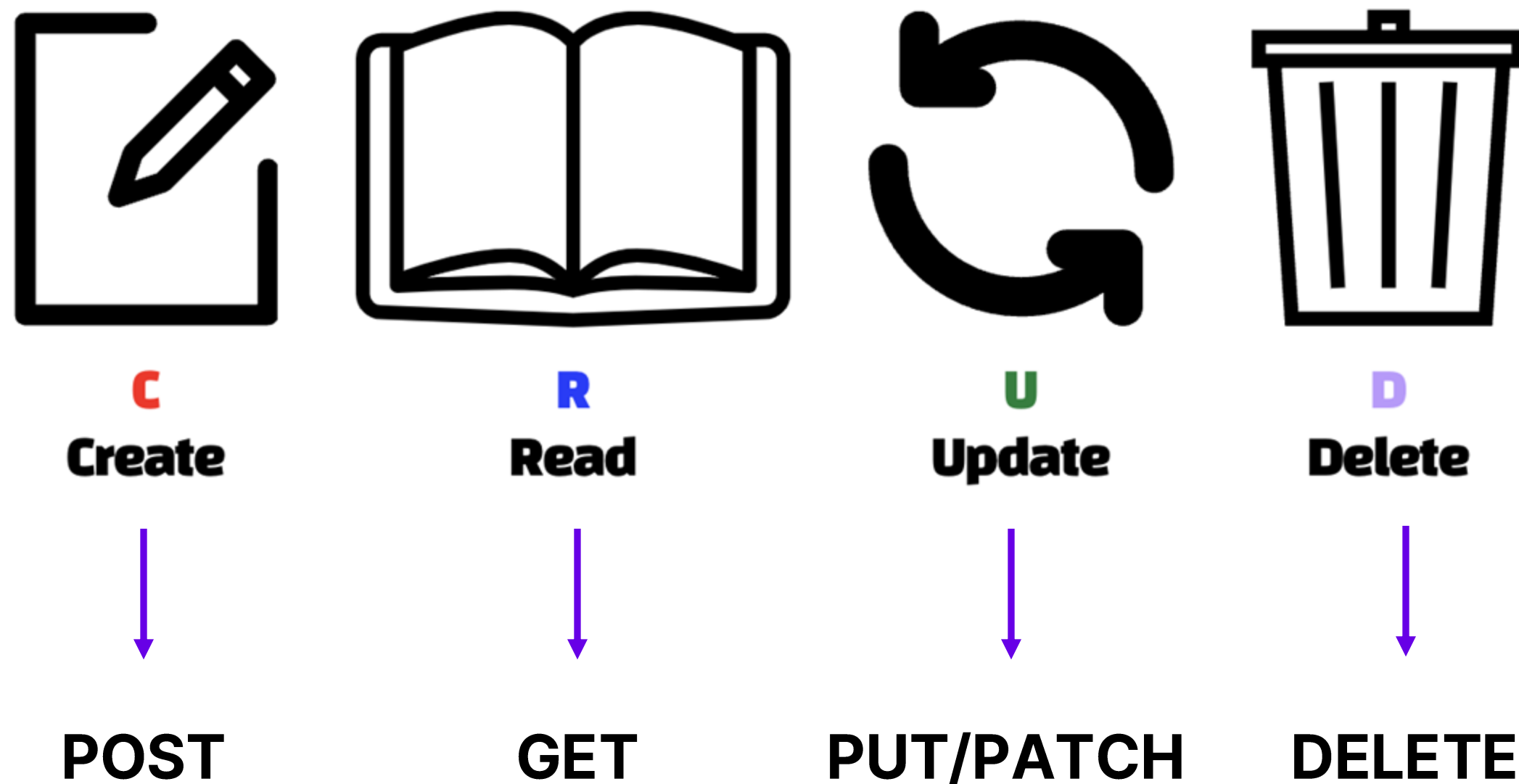


U
Update



D
Delete

주로 **CRUD(Create, Read, Update, Delete)** 작업을 표현



메서드	역할
GET	자료를 조회할 때 사용
POST	자료를 등록할 때 사용
PUT	자료의 수정을 요청할 때 사용
Delete	자료의 삭제를 요청할 때 사용

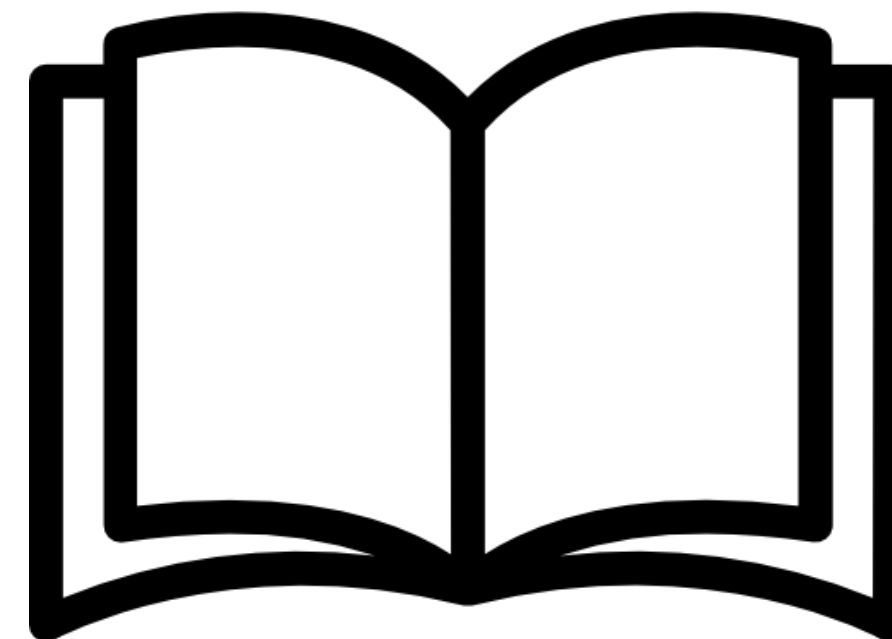
주로 **CRUD(Create, Read, Update, Delete)** 작업을 표현



GET 메서드

- 서버로부터 리소스를 요청하고 **데이터를 조회**
- 데이터 검색, 웹 페이지 로딩 등에 사용
- 요청 본문 X
- 요청 URL에 쿼리 파라미터*를 포함할 수 있음
 - `https://example.com/search?keyword=elice`

```
GET /api/users HTTP/1.1  
Host: example.com
```



READ

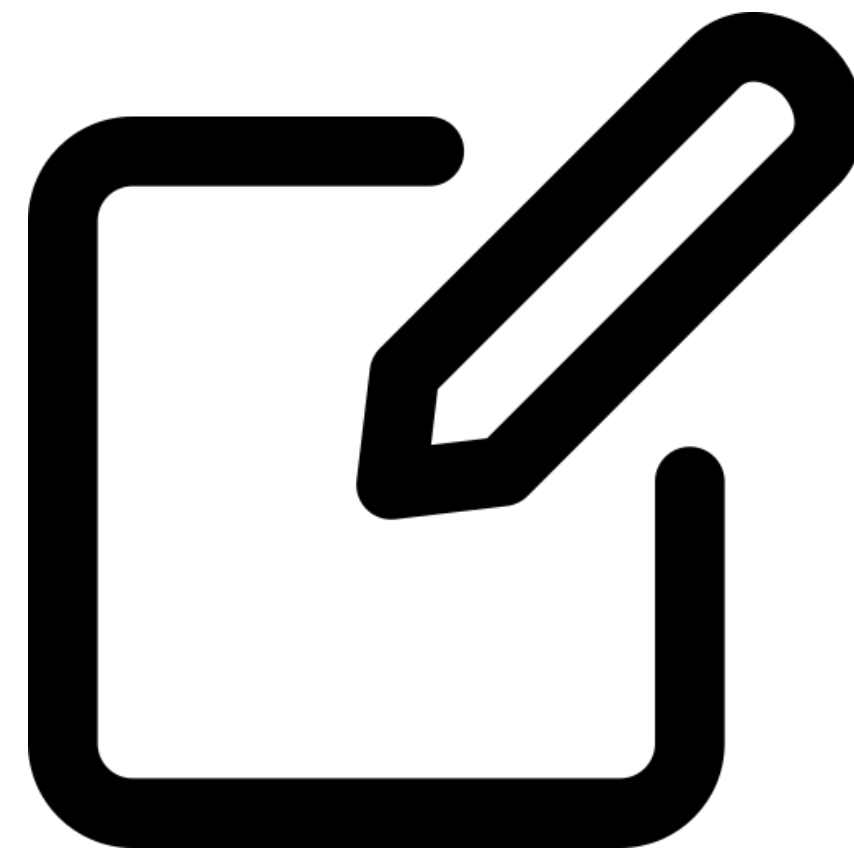


POST 메서드

- 서버에 데이터를 전송하여 **새로운 데이터를 생성**
- 폼 제출, 파일 업로드, 회원 가입 등에 사용
- 요청 본문에 데이터 포함
- 서버의 상태를 변경함

```
POST /api/users HTTP/1.1
Host: example.com
Content-Type: application/json

{
  "name": "John Doe",
  "email": "john.doe@example.com"
}
```



CREATE

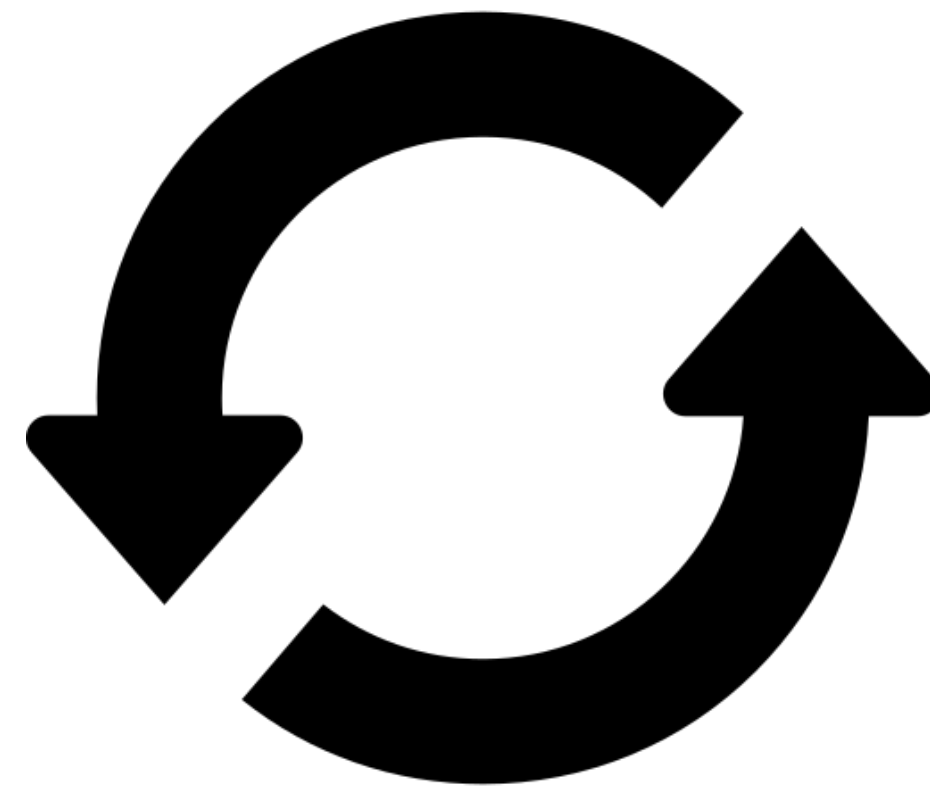


PUT 메서드

- 서버에 데이터를 전송하여 **리소스를 완전히 대체함**
- PATCH 메서드는 일부 대체
- 데이터 전체 갱신 시 사용
- 요청 본문에 데이터 포함

```
PUT /api/users/123 HTTP/1.1
Host: example.com
Content-Type: application/json
```

```
{
  "name": "John Doe",
  "email": "john.doe@example.com"
}
```



UPDATE



DELETE 메서드

- 서버에서 **특정 리소스를 삭제**
- 요청 본문 없음

```
DELETE /api/users/123 HTTP/1.1  
Host: example.com
```



DELETE



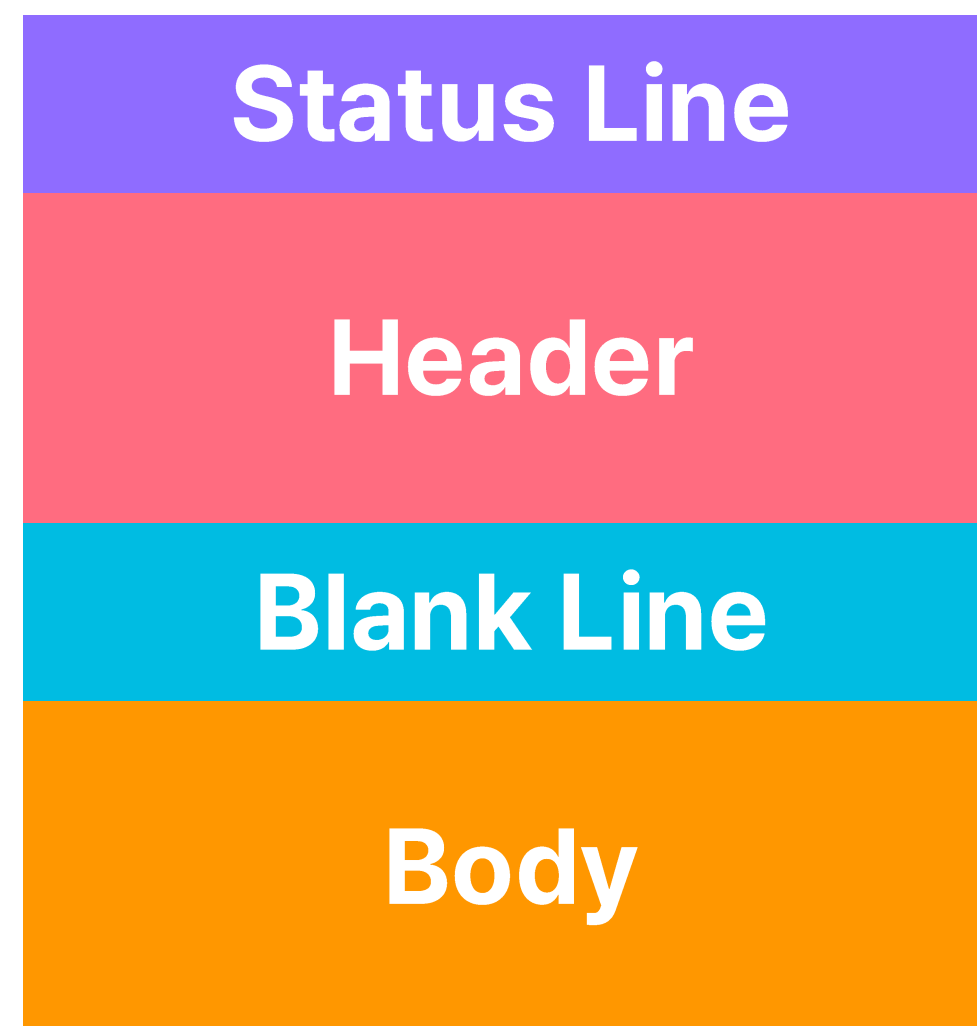
기타 메서드

- **HEAD 메서드**
 - GET 요청과 유사하지만 응답 본문을 포함 X
- **OPTIONS 메서드**
 - 서버가 지원하는 HTTP 메서드 목록 요청
- **TRACE 메서드**
 - 리소스에 대한 경로를 따라 메시지 루프백 메시지 수행

4. HTTP 요청 메시지/응답 메시지



요청 메시지

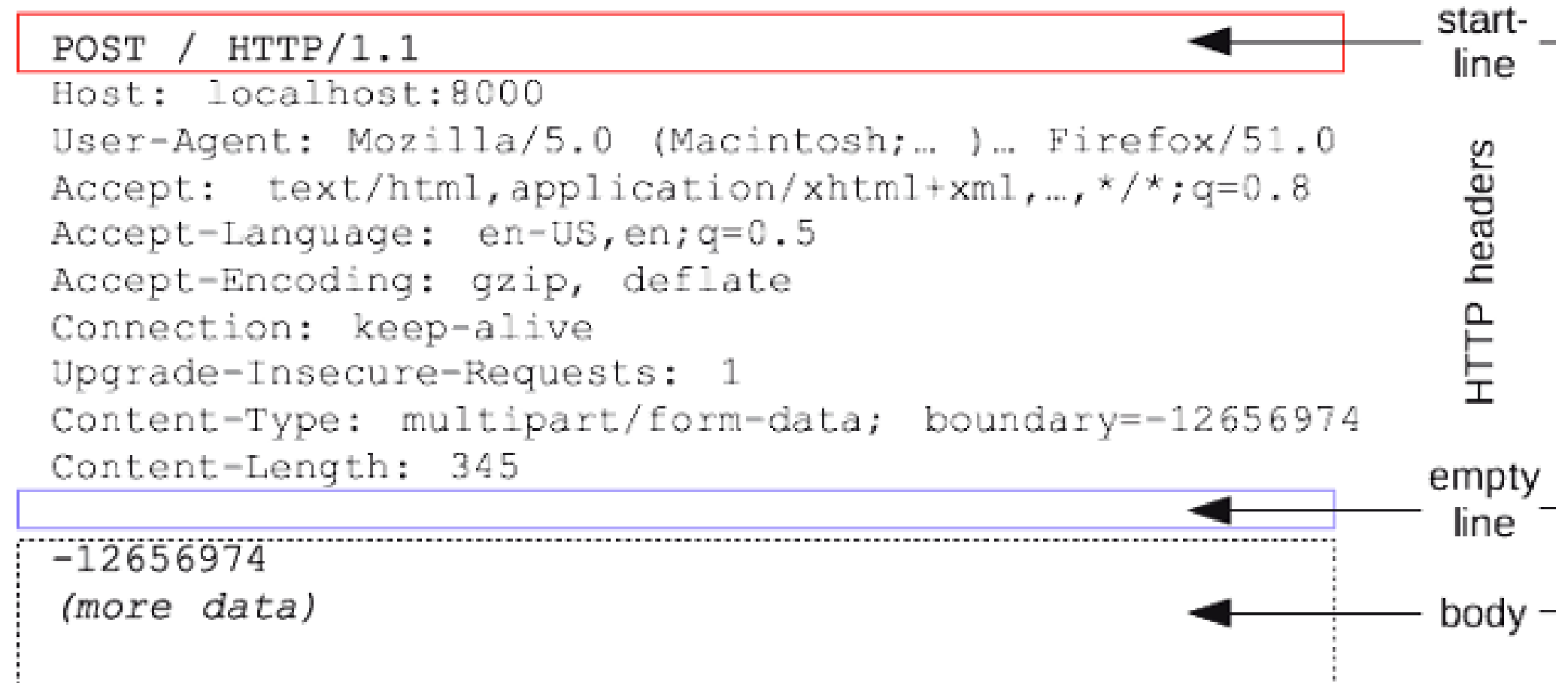


응답 메시지



요청 메시지

Requests





요청 메시지

```
POST /api/data HTTP/1.1
Host: www.example.com
User-Agent: Mozilla/5.0 (Windows NT 10.0; ...
Accept: application/json
Content-Type: application/json
Content-Length: 75

{
  "name": "John Doe",
  "email": "john.doe@example.com",
  "age": 30
}
```



요청 메시지

POST /api/data HTTP/1.1



요청 메시지

POST /api/data HTTP/1.1



HTTP 메서드

클라이언트가 서버에 요청하는 작업의 종류를 표현

메서드	역할
GET	자료를 조회할 때 사용
POST	자료를 등록할 때 사용
PUT	자료의 수정을 요청할 때 사용
Delete	자료의 삭제를 요청할 때 사용



요청 메시지

POST /api/data HTTP/1.1



요청 타겟

클라이언트가 접근하고자 하는 자원의 경로를 표시
주로 URL로 표현



요청 메시지

POST /api/data HTTP/1.1



HTTP 버전



요청 메시지

```
Host: www.example.com
User-Agent: Mozilla/5.0 (Windows NT 10.0; ...
Accept: application/json
Content-Type: application/json
Content-Length: 75
```



Header

헤더는 요청에 대한 부가 정보를 제공

헤더의 각 속성은 **이름-값 쌍**으로 설명되며 **콜론(:)**으로 구분

일반 헤더, 요청 헤더, 응답 헤더 등으로 구분



요청 메시지

```
Host: www.example.com
User-Agent: Mozilla/5.0 (Windows NT 10.0; ...
Accept: application/json
Content-Type: application/json
Content-Length: 75
```



Content-Type

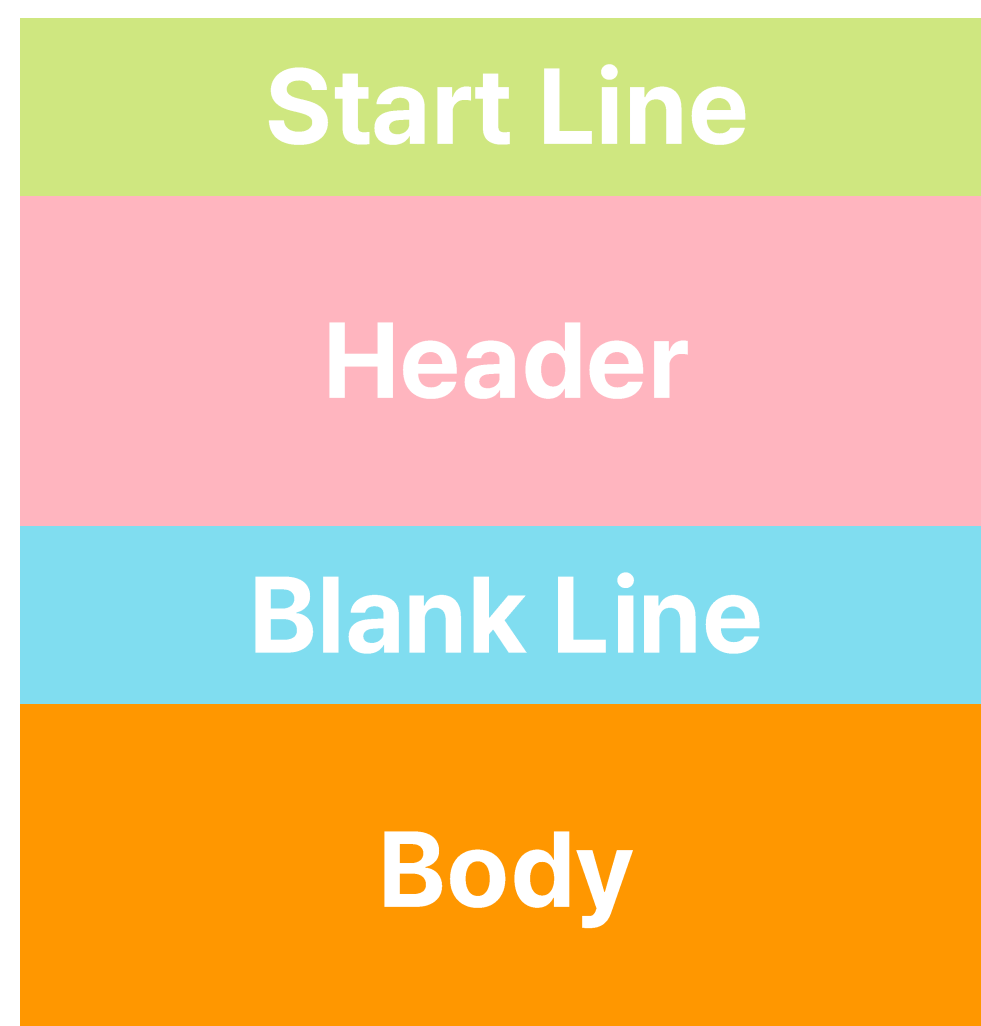
Body가 존재하는 경우, Body에 전달되는 데이터의 형식을 설명

ex) application/json, text/html



요청 메시지

- **Host:** 요청하는 호스트의 이름 및 포트 번호
- **User-agent:** 클라이언트가 사용하는 웹 브라우저 정보
- **Referer:** 현재 요청된 페이지로 이동하기 직전에 방문한 웹 페이지의 URL
- **Accept:** 클라이언트가 처리할 수 있는 데이터 타입 및 우선 순위
- **If-Modified-Since:** 값으로 지정한 날짜 이후에 수정된 자원 요청
- **Authorization:** 클라이언트가 가지고 있는 서버에 대한 권한 정보
- **Origin:** 요청이 시작된 주소 정보
- **Cookie:** 서버에 의해 설정된 클라이언트의 쿠키 정보



요청 메시지

Body

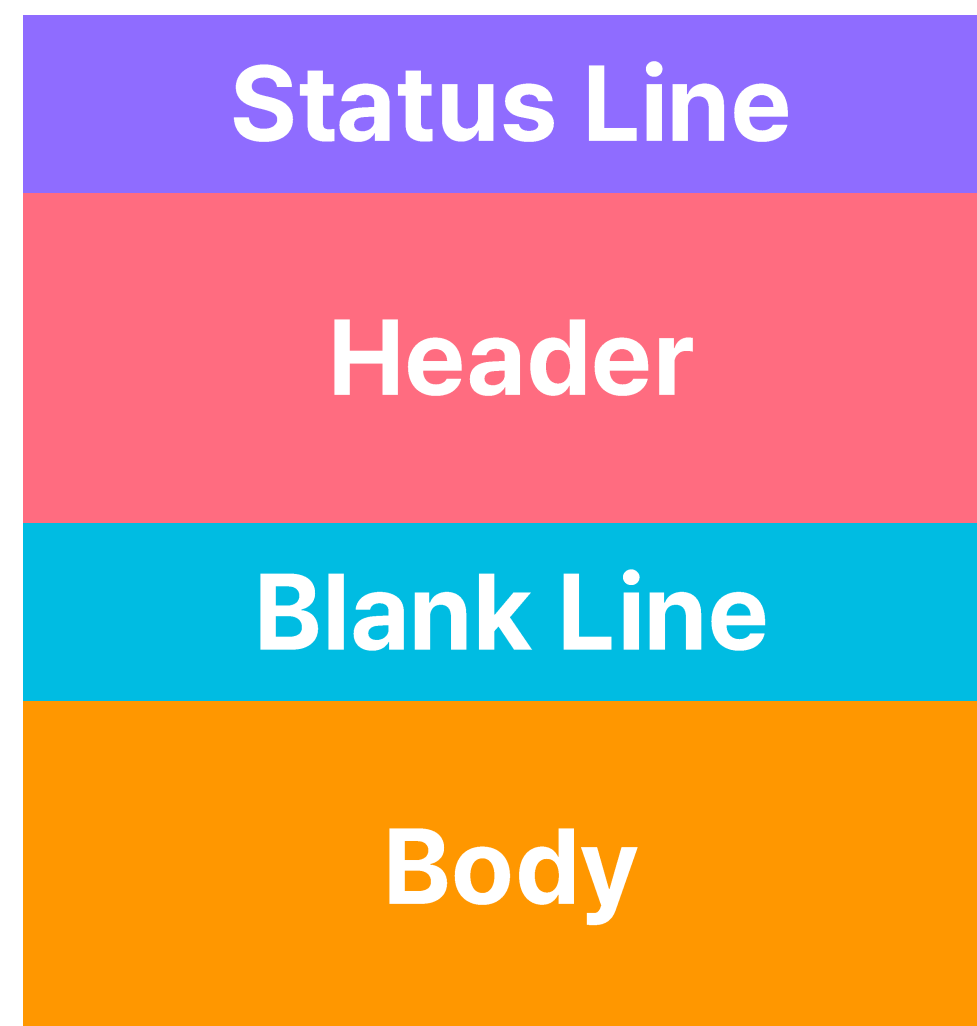
클라이언트가 서버에 전달하는 실제 데이터

주로 POST, PUT 요청에 사용

```
{  
  "name": "John Doe",  
  "email": "john.doe@example.com",  
  "age": 30  
}
```



요청 메시지



응답 메시지



Responses

start-line → HTTP/1.1 403 Forbidden

HTTP headers
Server: Apache
Content-Type: text/html; charset=iso-8859-1
Date: Wed, 10 Aug 2016 09:23:25 GMT
Keep-Alive: timeout=5, max=1000
Connection: Keep-Alive
Age: 3464
Date: Wed, 10 Aug 2016 09:46:25 GMT
X-Cache-Info: caching
Content-Length: 220

empty line →

body →
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML
2.0//EN">
(more data)

Status Line

Header

Blank Line

Body

응답 메시지



```
HTTP/1.1 200 OK
Date: Mon, 27 Jul 2023 12:28:53 GMT
Server: Apache/2.4.41 (Ubuntu)
Content-Type: application/json
Content-Length: 85
```

```
{
  "status": "success",
  "data": {
    "id": 123,
    "name": "John Doe",
    "email": "john.doe@example.com"
  }
}
```

Status Line

Header

Blank Line

Body

응답 메시지



HTTP/1.1 200 OK

Status Line

Header

Blank Line

Body

응답 메시지



HTTP/1.1 200 OK



HTTP 버전

HTTP 프로토콜 버전

Status Line

Header

Blank Line

Body

응답 메시지



HTTP/1.1 200 OK



HTTP 상태 코드

클라이언트의 요청의 상태에 대한 서버의 응답을
나타내는 3자리 숫자 코드

123

Status Line

Header

Blank Line

Body

응답 메시지



HTTP/1.1 200 OK



상태 메시지

응답 상태에 대한 설명

Status Line

Header

Blank Line

Body

응답 메시지



```
Date: Mon, 27 Jul 2023 12:28:53 GMT  
Server: Apache/2.4.41 (Ubuntu)  
Content-Type: application/json  
Content-Length: 85
```

Status Line

Header

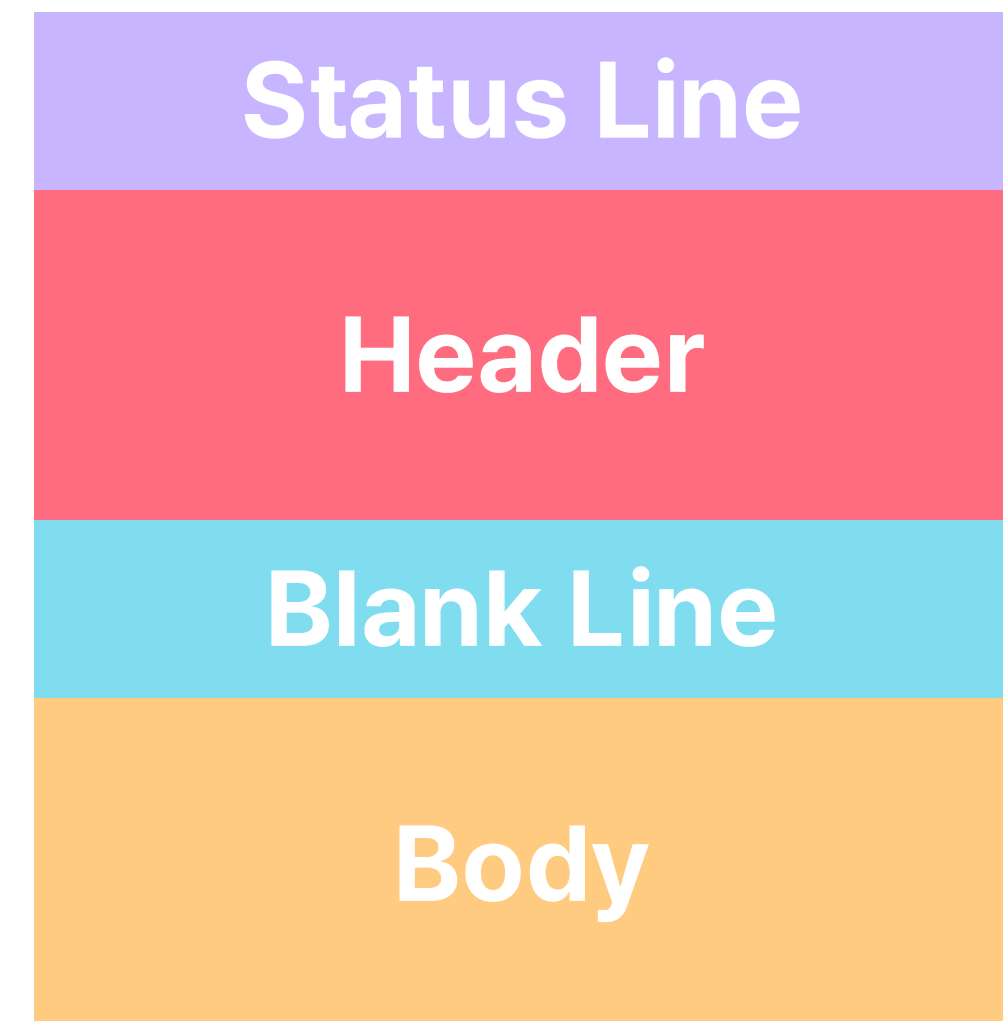
Blank Line

Body

응답 메시지

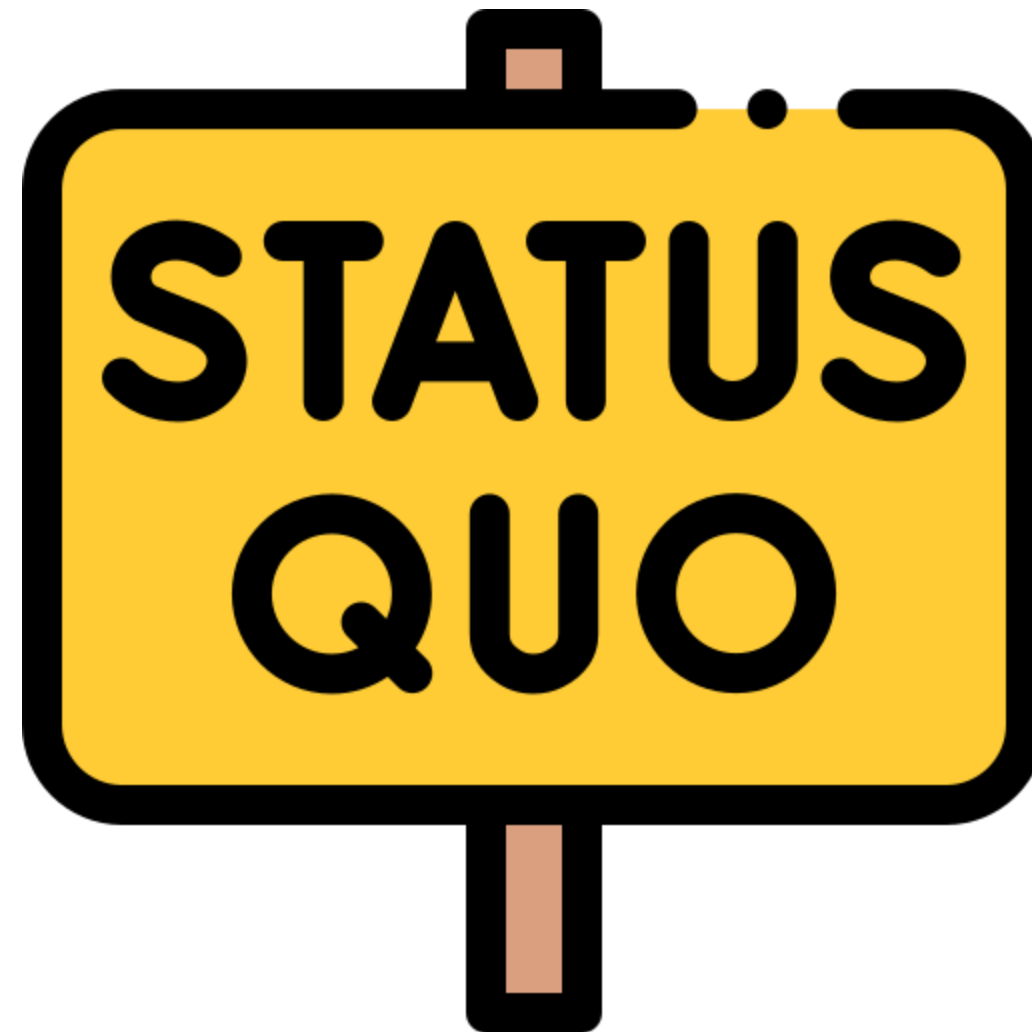


- **Location:** 리다이렉션 해야 하는 페이지의 URL
- **Server:** 서버에서 사용하는 소프트웨어 정보
- **Age:** 응답이 생성되고 지난 초 단위의 시간
- **Referrer-Policy:** 요청 메시지에 포함할 Referer 정보의 양을 제어
- **WWW-Authenticate:** 클라이언트가 서버에 인증해야 하는 방식
- **Proxy-Authenticate:** 클라이언트가 프록시 서버로 인증 정보를 보내는 방식



응답 메시지

5. HTTP 상태 코드



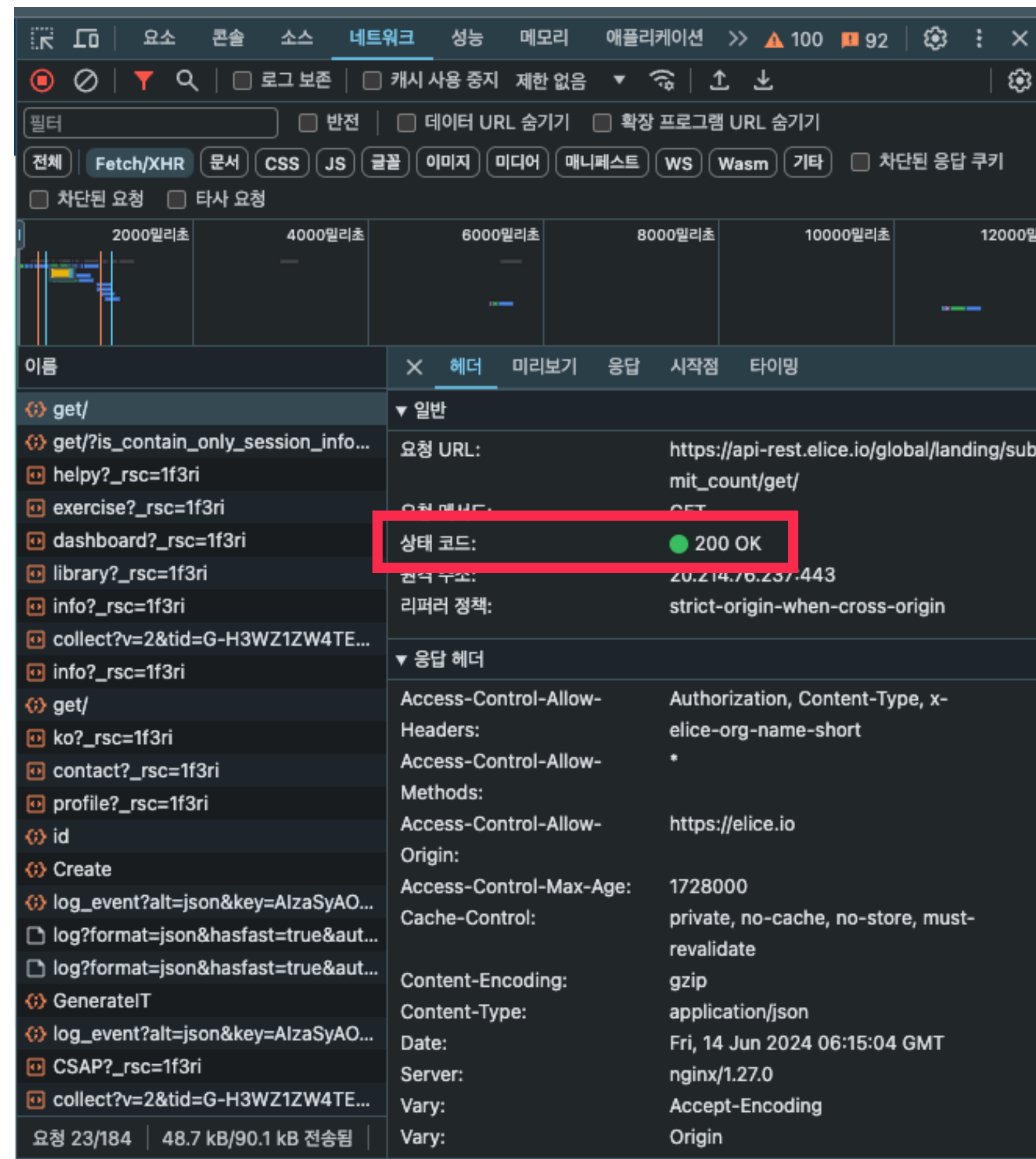
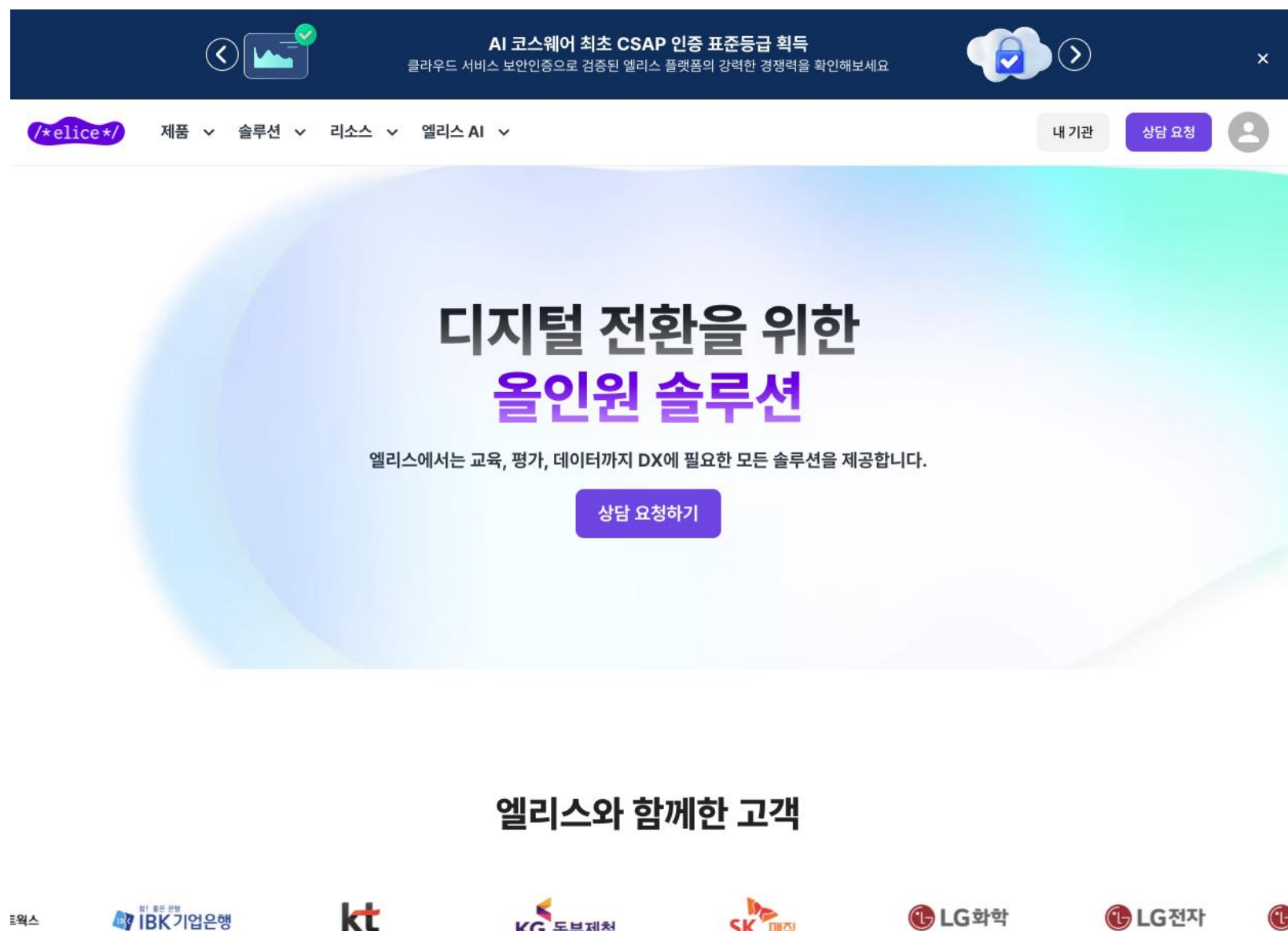
클라이언트의 요청에 대한 **서버의 응답을 나타내는 3자리 숫자 코드**



HTTP 상태 코드

02 HTTP 프로토콜과
RESTful API 기본

5. HTTP 상태 코드





- **1xx (정보 응답):** 요청을 처리 중임을 나타냄
- **2xx (성공):** 요청이 성공적으로 처리됨
- **3xx (리다이렉션):** 요청 완료를 위해 추가 작업이 필요함
- **4xx (클라이언트 오류):** 클라이언트의 요청이 잘못됨
- **5xx (서버 오류):** 서버에서 요청을 처리하지 못함





1xx 상태 코드 (정보)



코드	설명
100	Continue - 클라이언트가 요청의 첫 번째 부분을 전송했으며, 서버가 나머지 요청을 계속해서 전송하라고 응답
101	Switching Protocol - 클라이언트가 프로토콜 전환을 요청했으며, 서버가 이를 승인
102	Processing - 서버가 요청을 수신하여 처리 중이지만, 아직 완료되지 않았음을 나타냄
103	Checkpoint - 응답 헤더의 초기 힌트를 클라이언트에게 제공하여 리소스 로드를 가속화

“요청을 받았으며 프로세스를 계속함”



2xx 상태 코드 (성공)



코드	설명
200	OK - 요청이 성공적으로 처리되었음을 나타냄
201	Created - 요청이 성공적으로 처리되었으며, 새로운 리소스가 생성되었음을 나타냄
202	Accepted - 요청이 수락되었지만, 아직 처리되지 않았음을 나타냄
203	Non-Authoritative Information - 요청이 성공적으로 처리되었지만, 응답 헤더 정보가 원본 서버가 아닌 다른 소스로부터 왔음을 나타냄
204	No Content - 요청이 성공적으로 처리되었지만, 반환할 내용이 없음을 나타냄

“요청을 성공적으로 수신하고 실행함”



3xx 상태 코드 (리다이렉션)



코드	설명
300	Multiple Choices - 요청한 리소스에 대한 여러 가지 선택지가 있음을 나타냄
301	Moved Permanently - 요청한 리소스가 영구적으로 새로운 URL로 이동되었음을 나타냄
302	Found - 요청한 리소스가 임시로 다른 URL에 위치함을 나타냄
304	Not Modified - 요청한 리소스가 수정되지 않았으며, 클라이언트의 캐시된 버전을 사용할 수 있음을 나타냄

“요청 완료를 위해 추가 작업 조치 필요”



4xx 상태 코드 (클라이언트 오류)

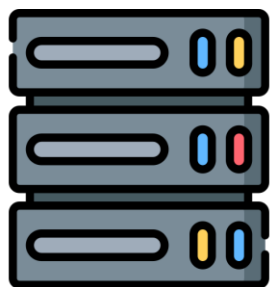


코드	설명
400	Bad Request - 클라이언트의 요청이 잘못되었음을 나타냄
401	Unauthorized - 인증이 필요하지만 제공되지 않았거나 유효하지 않음을 나타냄
403	Forbidden - 서버가 요청을 이해했지만, 권한이 없어서 요청을 거부함
404	Not Found - 요청한 리소스를 찾을 수 없음을 나타냄
405	Method Not Allowed - 요청한 메서드가 허용되지 않음을 나타냄

“클라이언트의 요청 오류”



5xx 상태 코드 (서버 오류)



코드	설명
500	Internal Server Error - 서버가 요청을 처리하는 도중 예기치 않은 상태에 도달했음을 나타냄
501	Not Implemented - 서버가 요청을 수행할 방법을 지원하지 않음을 나타냄
502	Bad Gateway - 게이트웨이 또는 프록시 서버가 상위 서버로부터 잘못된 응답을 받았음을 나타냄
503	Service Unavailable - 서버가 일시적으로 요청을 처리할 수 없음을 나타냄

"서버의 응답 오류"

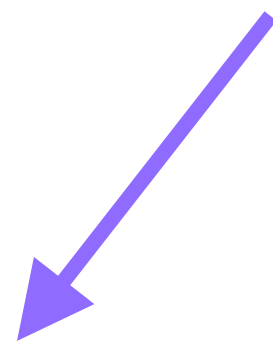
6. REST API 개요



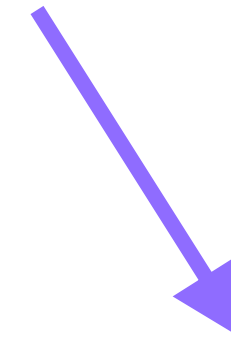
REST API



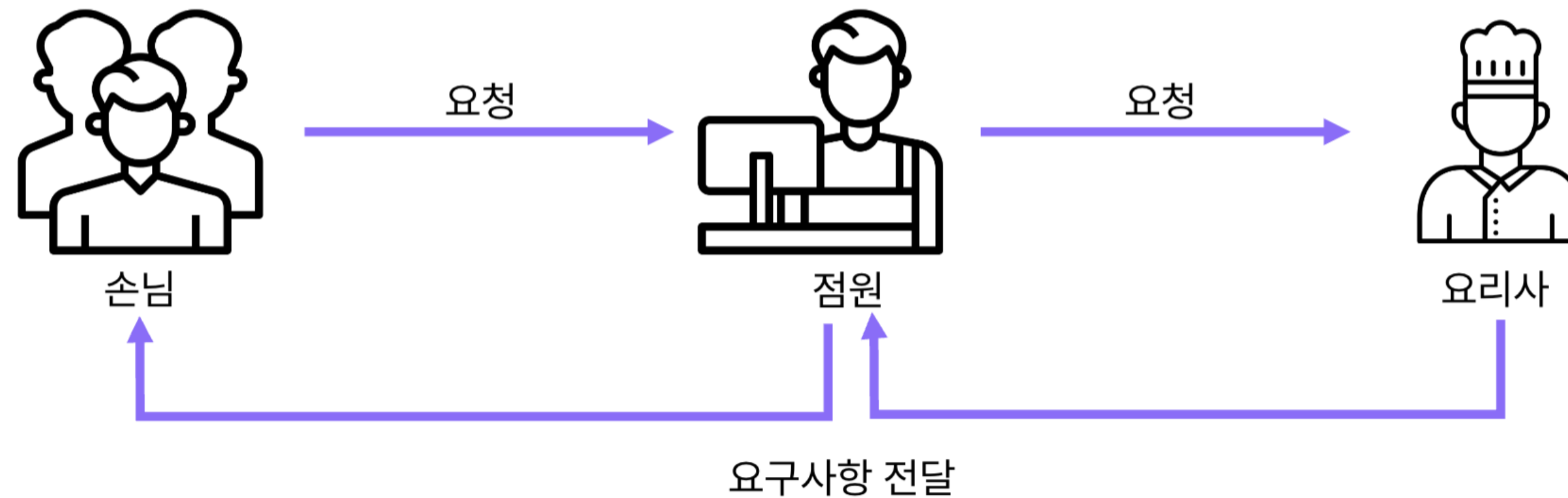
REST + API



- **R**epresentational **S**tate **T**ransfer
- HTTP 프로토콜에 기반하여 자원을 관리하는 방식을 정의
- 자원을 이름으로 식별하여 자원의 상태 정보를 주고 받음



- **A**pplication **P**rogramming **I**nterface
- 시스템 간 **상호작용을 위한 인터페이스**
- 레스토랑의 점원



HTTP 프로토콜에 기반하여 “**REST 원칙을 따르는 API**”



REST API



1. 클라이언트-서버(Client-Server) 구조

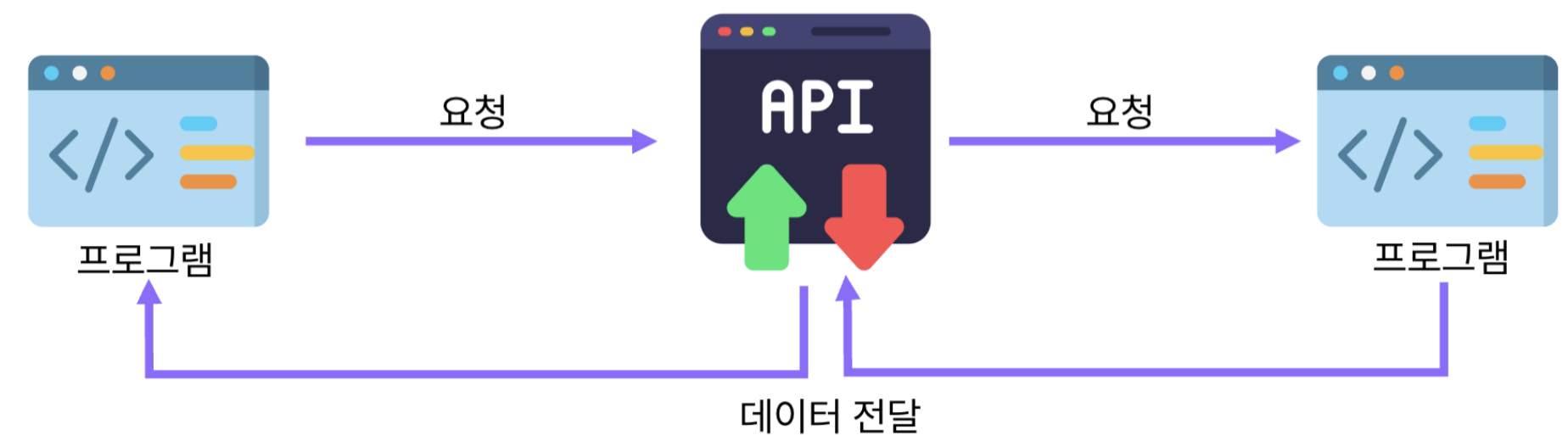
- 사용자 인터페이스와 비즈니스 로직/데이터 저장의 분리

2. 무상태(Stateless)

- 각 요청은 독립적, 서버에 상태 정보 유지 X

3. 캐시 처리 가능(Cacheable)

- 응답은 캐싱 가능해야 함





4. 계층화 시스템(Layered System)

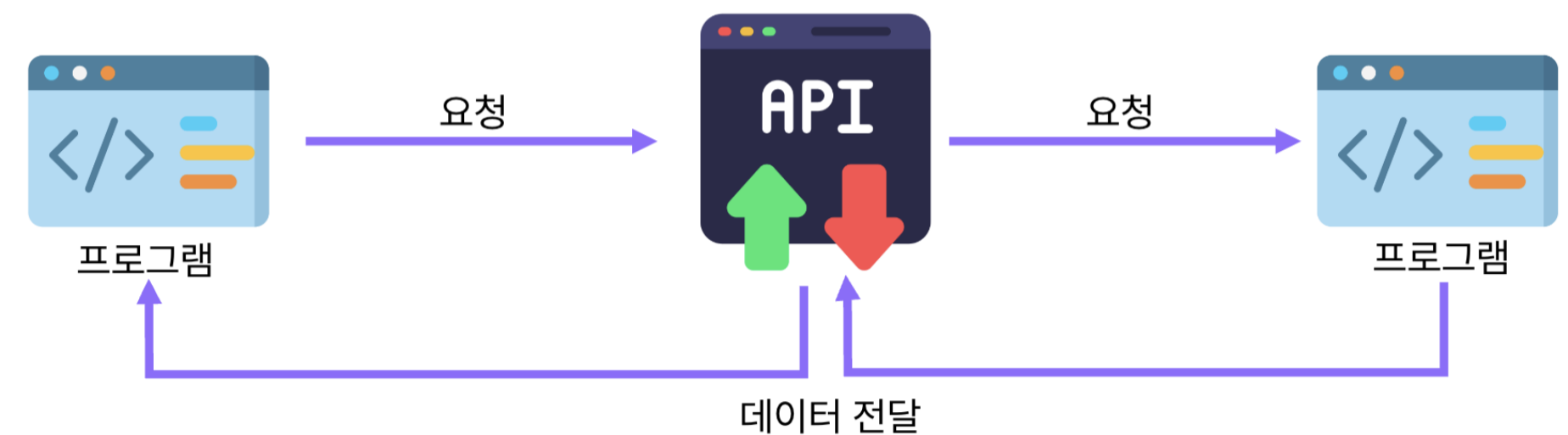
- 클라이언트는 서버를 직접 호출하지 않음 -> 다중 계층 허용 (로드 밸런싱, 보안, 암호화 등)

5. 인터페이스 일관성(Uniform Interface)

- 효율적인 아키텍처를 위한 일관된 인터페이스

6. 코드 온 디맨드(Code-On-Demand)

- 서버는 실행 가능 코드 형태로 기능을 전송할 수 있음





RESTful

7. REST API의 구성요소



자원

행위

표현



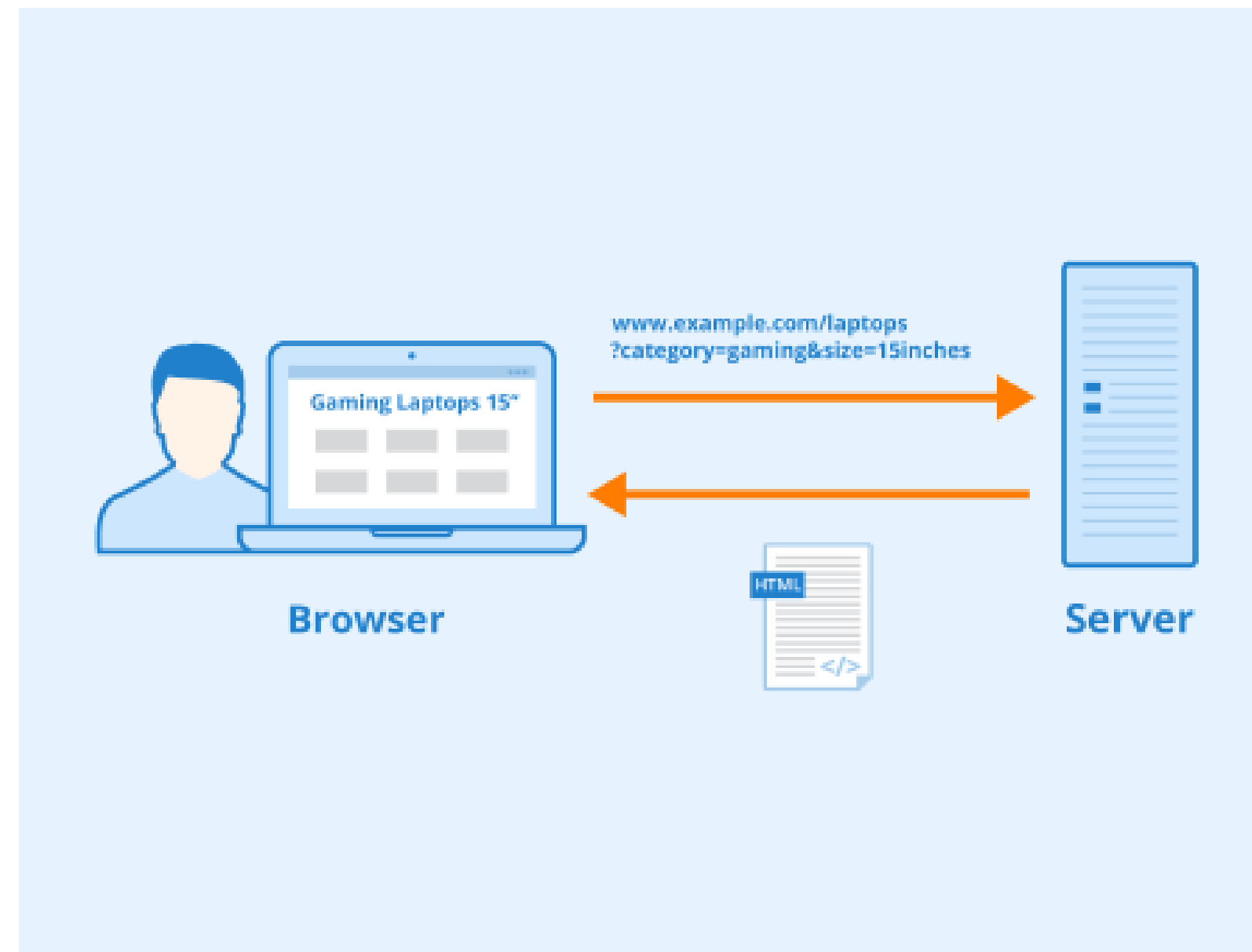
자원

1. 자원(Resources) – URI

- 모든 자원에 고유한 Id가 존재
- 데이터베이스의 엔티티나 정보의 단위
 - 사용자, 제품, 주문 등



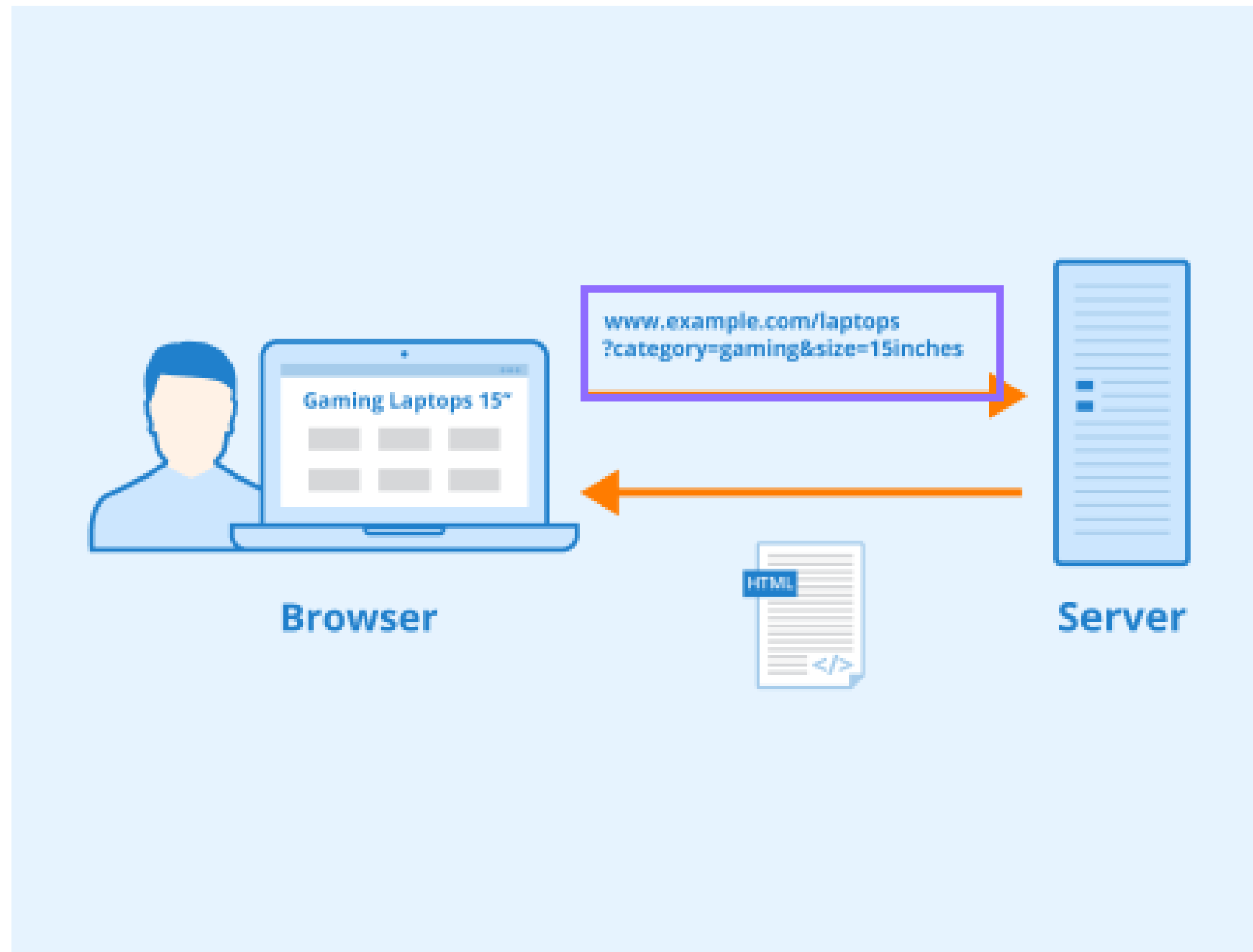
1. 자원(Resources) – URI



- Uniform Resource Identifier
- URI는 정보의 자원을 표현함; 자원 식별자



1. 자원(Resources) – URI





1. 자원(Resources) – URI





1. 자원(Resources) – URI



- **Scheme:** 리소스에 접근하기 위한 프로토콜
- **Host:** 리소스를 호스팅하는 서버 이름 또는 IP
- **Port:** 서버에서 리소스에 접근하기 위해 사용되는 게이트
- **Path:** 호스트 내에서 리소스의 정확한 위치
- **Query Parameters:** 리소스에 전달되는 추가정보
- **Fragment Identifier:** 부가적인 식별자



행위

2. 행위(Verbs) - HTTP 메서드

- GET, POST, PUT, DELETE 등



2. 행위(Verbs) - HTTP 메서드



C
Create



R
Read



U
Update



D
Delete

메서드	역할
GET	자료를 조회할 때 사용
POST	자료를 등록할 때 사용
PUT	자료의 수정을 요청할 때 사용
Delete	자료의 삭제를 요청할 때 사용

- 자원에 대한 조작을 정의함
- GET(조회), POST(등록/생성), PUT(수정), DELETE(삭제)



2. 행위(Verbs) - HTTP 메서드

메서드	역할
[GET] /users/{userId}	특정 사용자의 상세 정보를 조회
[POST] /users	특정 사용자를 등록
[PUT] /users/{userId}	특정 사용자의 정보를 수정
[DELETE] /users/{userId}	특정 사용자의 정보를 삭제



1. 리소스명은 동사보다는 명사를 사용한다.
2. 리소스에 대한 행위는 HTTP 메서드로 표현한다.
3. URL 경로에는 소문자를 사용한다.
4. 계층 관계는 슬래시(/)로 구분한다.
5. 마지막 문자로 슬래시(/)를 사용하지 않는다.
6. 의미 구분을 위해서는 하이픈(-)을 사용하고 언더바(_)를 사용하지 않는다.

RESTful



표현

3. 표현(Representations) - HTTP 메시지 페이로드

- 클라이언트가 자원의 상태에 대한 조작을 요청 -> 서버 응답
- 자원은 JSON, XML, TEXT 등 여러 형태를 가질 수 있음



3. 표현(Representations) - HTTP 메시지 페이로드

animals.json

```
[  
  { name: 'cat', legs: 4 },  
  { name: 'chicken', legs: 2 },  
]
```

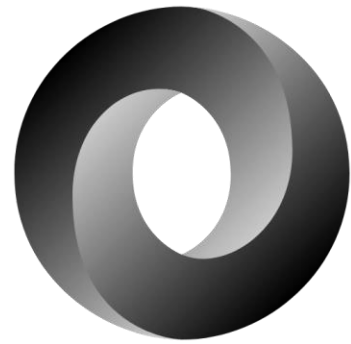
animals.xml

```
<array>  
  <item>  
    <name>cat</name>  
    <legs>4</legs>  
  </item>  
  <item>  
    <name>chicken</name>  
    <legs>2</legs>  
  </item>  
</array>
```

- 가장 대표적인 데이터 포맷 2 가지
 - **JSON(Javascript Object Notation)**과 **XML(eXtensible Markup Language)**
- 최근에는 경량이고 가독성이 높은 **JSON 포맷**을 보편적으로 사용



3. 표현(Representations) – JSON 객체

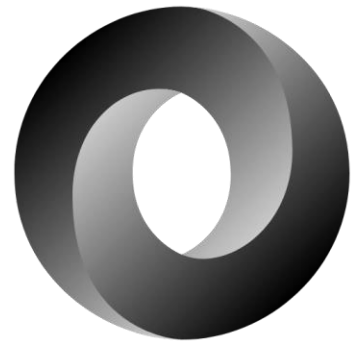


```
{
  "name": "John Doe",
  "age": 30,
  "isEmployee": true,
  "address": {
    "street": "123 Main St",
    "city": "Anytown"
  },
  "phoneNumbers": ["123-456-7890", "456-789-0123"]
}
```

- 경량의 데이터 교환 형식, 프로그래밍 언어 독립적
- 키/값 쌍의 집합 구조



3. 표현(Representations) – JSON 배열



```
[  
  {  
    "name": "John Doe",  
    "age": 30  
  },  
  {  
    "name": "Jane Smith",  
    "age": 25  
  }  
]
```

- 순서 있는 값의 리스트
- 각 값은 JSON 객체가 될 수 있음



클라이언트

POST /posts

```
{  
  name: "New Post",  
  content: "새로운 게시물 내용입니다."  
}
```



서버





HTTP 메서드 = 행위

POST /posts

```
{  
  name: "New Post",  
  content: "새로운 게시물 내용입니다."  
}
```





1. 자원 식별

- 인터넷에 있는 모든 것은 **고유한 주소(URI)**를 가져야 함
- 자원이 URI로 식별되어야 함, 특정 자원은 오직 하나의 URL에 매핑되어야 함
- [예시]
 - <https://mywebsite.com/profiles/john> -> John의 프로필 페이지

2. 표현을 통한 자원의 조작

- 웹 페이지의 정보를 조작할 때 해당 정보는 XML, JSON 같은 형식으로 구성됨
- 이 같이 특정 형식을 통해 정보에 대한 조회, 수정, 삭제 등을 수행해야 함
- URL을 바꾸지 않고도 표현을 활용해서 리소스를 조작 가능
- [예시]: JSON 형식
 - {"name": "Jane", "age": "30"}



3. 자기 서술적 메시지

- 메시지가 자기 자신을 어떻게 처리해야 할지 설명해야 함
- 수신하는 쪽에서 별도의 설명 없이도 해당 메시지를 이해하고 처리할 수 있도록 정의
- [예시]
 - 'Content-Type: text/html' => 해당 메시지가 HTML 형식의 텍스트를 포함한다는 사실 알 수 있음

4. HATEOAS(Hypermedia As The Engine of Application State)

- 웹 페이지가 다른 페이지나 상태로 넘어갈 수 있도록 링크를 제공해야 함
- 링크를 통해 사용자나 프로그램이 다음에 할 수 있는 동작을 유추 가능
- [예시]
 - ' 다음 페이지' '이전 페이지'로 가는 링크를 제공